

Computing conditional probabilities in Bayesian networks using logistic regression

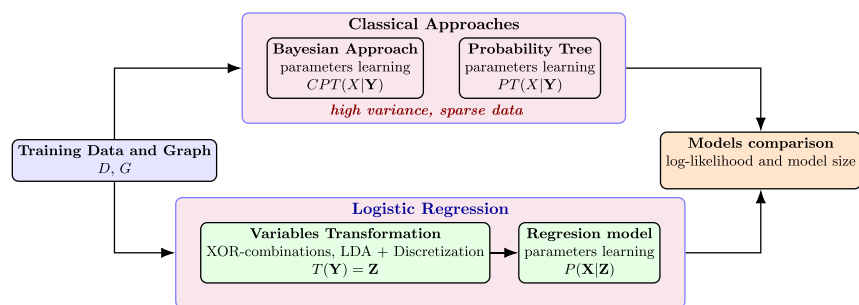
Serafín Moral, Serafín Moral-García*, Andrés Cano , Manuel Gómez-Olmedo

Department of Computer Science and Artificial Intelligence, University of Granada, 18071, Granada, Spain

HIGHLIGHTS

- In Bayesian networks, parameters grow exponentially with the number of parents.
- Enhanced logistic regression uses synthetic variables to improve model flexibility.
- New variables are created via XOR combinations and LDA-based discretization.
- The paper compares transformations via completeness, equivalence, and subsumption.
- LDA-based logistic regression excels with fewer parameters than traditional tables.

GRAPHICAL ABSTRACT



ARTICLE INFO

Keywords:

Conditional probabilities
Logistic regression
Bayesian networks
Linear discriminant analysis
Classification trees

ABSTRACT

This study addresses the estimation of conditional probability tables in Bayesian networks. Traditional procedures encounter difficulties when the number of parent variables is large because the number of parameters grows exponentially. Logistic regression can mitigate this dimensionality challenge, but it imposes assumptions about how a variable depends on its parents. To extend its applicability to more general scenarios, we introduce a method for systematically constructing artificial variables derived from the original predictors. We develop a general theoretical framework to compare alternative formulations of these variables. Two main approaches are proposed. The first relies on XOR combinations of the original variables, whereas the second is based on Linear Discriminant Analysis and discretization. The proposed approach is validated through an extensive empirical evaluation on large- and medium-scale networks, where we benchmark logistic regression against established techniques, including Bayesian inference procedures and probability tree models. The experiments demonstrate that logistic regression and discretization yield excellent performance while requiring fewer parameters than a full conditional probability table.

1. Introduction

Bayesian networks [1,2] are probabilistic graphical models comprising two principal components: a qualitative structure, represented by

a directed acyclic graph (DAG), and a quantitative component consisting of the conditional probability distributions of each variable given its parents in the graph. When learning a Bayesian network from data, the most challenging task is typically identifying the network structure,

* Corresponding author.

Email addresses: smc@decsai.ugr.es (S. Moral), seramoral@decsai.ugr.es (S. Moral-García), acu@decsai.ugr.es (A. Cano), mgomez@decsai.ugr.es (M. Gómez-Olmedo).

<https://doi.org/10.1016/j.asoc.2026.115284>

Received 12 November 2025; Received in revised form 9 April 2026; Accepted 10 April 2026

Available online 19 April 2026

1568-4946/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

while the conditional probabilities can be estimated using standard statistical methods [3], such as maximum likelihood estimation or Bayesian inference. However, a well-known limitation is that the size of a conditional probability table (CPT) grows exponentially with the number of parent variables, which can lead to sparse—or even empty—data subsets for parameter estimation [4].

To mitigate this issue, several models have been proposed to reduce the number of parameters required to represent the dependence of a variable on its parents. Notable examples include noisy-OR gates [5,6], logistic regression models [7,8], and probability trees (PT) [9–11]. Noisy-OR gates require an additional assumption of causal independence, restricting their applicability to specific contexts, whereas logistic regression is fundamentally constrained by the assumption of monotonicity in the relationships among variables [12]. Probability trees, in contrast, exploit asymmetric conditional independence relationships to simplify conditional probability representations. Although PTs are theoretically capable of representing any conditional probability distribution, their size can become comparable to that of a full CPT when no exploitable independence structure is present in the data.

In this paper, we propose using logistic regression as a general-purpose approach for estimating conditional probability distributions in Bayesian networks. As previously stated, logistic regression assumes the monotonicity hypothesis; consequently, its performance may deteriorate when this assumption is violated. The central hypothesis of this paper is that logistic regression constitutes a general and effective procedure for estimating conditional probabilities, provided that the feature space is augmented with artificial variables functionally derived from the original predictors. By doing so, we aim to enhance the flexibility and broaden the applicability of logistic regression beyond its conventional scope. We show that, under this framework, it can provide accurate estimates across a wide range of scenarios. Specifically, we consider two categories of artificial variables: simple logical combinations of existing variables and variables generated by classical discriminant analysis techniques [13] and discretization. The paper shows that this latter approach produces models with fewer parameters and very satisfactory performance on standard benchmark networks.

The procedures described here involve generating new variables derived from the current set of parents of a given variable X , denoted as $pa(X) = \mathbf{Y}$. Formally, this is achieved by applying a generic transformation T that yields a new set of variables $\mathbf{Z} = T(\mathbf{Y})$, such that the conditional probability distribution of X depends on the values of the transformed variables. We also provide a theoretical analysis of these transformations, introducing and formalizing the notions of completeness, equivalence, and subsumption.

The remainder of the paper is organized as follows. Section 2 reviews relevant literature and related work. Section 3 formulates the problem, establishes the notation used throughout, and reviews basic methods for conditional probability estimation. Section 4 presents our proposed logistic regression-based methodology, detailing the theoretical framework for feature transformations and the two specific transformation strategies introduced in this work. Section 5 discusses the results of an extensive experimental evaluation conducted to assess the performance of the proposed approach. Finally, Section 6 concludes the paper and outlines potential avenues for future research.

2. Related work

In addition to the probability tree and generalized Noisy-OR gate approaches mentioned in Section 1, several other research lines are directly relevant to the current study:

- Increasing the size of the training dataset by generating artificial cases, as proposed in [14]. We think that this approach is valid when the sample size is small, but not when the number of parents is too large since it would be necessary to enlarge the learning dataset to an exponential size as a function of the number of parents.

- Deeva and Bubnova [15] consider the use of logistic regression to define conditional probabilities. However, their work focuses on defining the conditional distribution of a discrete variable with continuous parents, and they employ standard logistic regression without introducing additional variables.
- Bengio and Bengio [16] propose estimating conditional probabilities using deep learning. The use of neural networks to model probabilistic relationships has received greater attention in the case of continuous variables (see Lachapelle et al. [17] or Kostic et al. [18]). Motomura and Hara [19] propose using neural networks when a node has both continuous and discrete variables.
- The problem of dimensionality can also be tackled by estimating a high order conditional probability by combining several low order distributions, as in [4]. This approach has been mainly developed in the scope of knowledge acquisition from experts to minimize the number of queries necessary to build a Bayesian network [20]. A comprehensive review of similar procedures is provided in [21]. Furthermore, the generalization of Noisy-OR models to causal independence interactions was proposed by Meek and Heckerman [22]. More recently, Drury et al. [23] proposed a recent relaxation of conditional independence assumptions that incorporates the construction of instrumental deterministic variables.
- A related challenge involves utilizing Bayesian networks to address queries with missing evidence values [24]. This approach estimates conditional probabilities for missing values based on observed data by leveraging Generative Adversarial Networks.

3. Problem statement and notation

Consider the qualitative structure of a Bayesian network defined by a set of discrete variables $\mathbf{X} = \{X_1, \dots, X_n\}$ and a directed acyclic graph (DAG) G . Discrete variables are represented by uppercase letters, such as X , and the set of possible values that X can take is Ω_X , where $|\Omega_X|$ is its cardinality. Sets of variables are written in bold uppercase (e.g., $\mathbf{X}$). A specific assignment of values to all variables in a set is called a *configuration*, written in bold lowercase (e.g., $\mathbf{x}$). The cardinality of a set $\mathbf{X}$ is denoted by $|\mathbf{X}|$. The set of all possible configurations of $\mathbf{X}$ is $\Omega_{\mathbf{X}}$, where $|\Omega_{\mathbf{X}}|$ is the total number of configurations.

The parents of a variable X_i in the graph are referred to as $pa(X_i)$. We assume the availability of a dataset $\mathcal{D}$ containing observations over the variables in the network. The problem is to estimate the conditional probability distribution of each variable X_i given its parents $pa(X_i)$, written as $P(X_i | pa(X_i))$. A central challenge arises from the fact that the number of possible configurations grows exponentially with the number of variables in the set.

For each value x_k of X_i and each configuration $\mathbf{y}_j$ of its parent set $pa(X_i) = \mathbf{Y}$, the corresponding conditional probability $P(X_i = x_k | \mathbf{Y} = \mathbf{y}_j)$ is represented by θ_{ijk} , where $i = 1, \dots, n$, $j = 1, \dots, |\Omega_{\mathbf{Y}}|$, and $k = 1, \dots, |\Omega_{X_i}|$. The size of a CPT $P(X_i | \mathbf{Y})$ is determined by the number of free parameters, which depends on the number of states of the child variable and the number of configurations of its parent set:

$$(|\Omega_{X_i}| - 1)|\Omega_{\mathbf{Y}}| = (|\Omega_{X_i}| - 1) \prod_{Y_j \in \mathbf{Y}} |\Omega_{Y_j}|$$

since for each pair (i, j) , $\sum_k \theta_{ijk} = 1$. In this same setting, let N_{ijk} be the number of occurrences in $\mathcal{D}$ where $X_i = x_k$ and $\mathbf{Y} = \mathbf{y}_j$, and define:

$$N_{ij} = \sum_{k=1}^{|\Omega_{X_i}|} N_{ijk}$$

as the number of occurrences of $\mathbf{Y} = \mathbf{y}_j$ in the dataset. The total number of cases in $\mathcal{D}$ is then:

$$N = \sum_{j=1}^{|\Omega_{\mathbf{Y}}|} N_{ij}.$$

Two standard approaches for parameter estimation in Bayesian networks are maximum likelihood and Bayesian estimation [2]. Under the

maximum likelihood approach, each parameter θ_{ijk} is estimated as:

$$\hat{\theta}_{ijk} = \frac{N_{ijk}}{N_{ij}}.$$

The Bayesian approach assumes a Dirichlet prior $D(\alpha_1, \dots, \alpha_r)$ for each parameter vector $(\theta_{ij1}, \dots, \theta_{ij|\Omega_{X_i}|})$. The posterior mean estimate is then:

$$\hat{\theta}_{ijk} = \frac{N_{ijk} + \alpha_i}{N_{ij} + s_i}, \quad (1)$$

where $s_i = |\Omega_{X_i}| \alpha_i$. When $\alpha_i = 1$, this corresponds to Laplace smoothing. More generally, a global sample size S is often used, with:

$$\alpha_i = \frac{S}{|\Omega_Y| \cdot |\Omega_{X_i}|} \quad s_i = \frac{S}{|\Omega_Y|}$$

The primary drawback of this procedure is the high dimensionality of the parameter space, which grows exponentially with the number of parent variables. Specifically, when the number of parents is large, a conditional probability for X_i must be estimated for each possible configuration y_j of its parents Y . The effective sample size for each estimation, denoted as N_{ij} , is determined by the number of instances in the training set that match the specific configuration y_j . Consequently, in many scenarios, N_{ij} will be extremely small or even zero—a situation that occurs whenever a configuration y_j is not represented in the learning dataset. This leads to severe overfitting and results in poor predictive performance when the model is applied to unseen data.

For structure learning, the Bayesian score is a common model selection criterion [25]. Let $Y = pa(X_i)$, $L = |\Omega_Y|$ be the number of parent configurations, and $r = |\Omega_{X_i}|$ the number of states of X_i . The Bayesian Dirichlet equivalent (BDE) score for X_i is:

$$BDE(X_i|Y, D, S) = \prod_{j=1}^L \left(\frac{\Gamma(S/L)}{\Gamma(N_{ij}) + S/L} \prod_{k=1}^r \frac{\Gamma(N_{ijk} + S/(Lr))}{\Gamma(S/(Lr))} \right), \quad (2)$$

where S is the global sample size and $\Gamma(x)$ is the gamma function. In practice, the logarithmic form $LBDE(X_i|Y, D, S)$ is often used.

Another widely used criterion is the Bayesian Information Criterion (BIC):

$$BIC(X_i|Y, D) = \sum_{j=1}^L \sum_{k=1}^r \frac{N_{ijk}}{N_{ij}} \log \left(\frac{N_{ijk}}{N_{ij}} \right) - \frac{1}{2}(r-1)L. \quad (3)$$

3.1. Probability trees

A probability tree (PT) is a data structure used to store CPTs, proving particularly effective in scenarios that involve context-specific conditional independence relationships [9–11]. Formally, a PT is a tree composed of internal and terminal nodes. Consider a tree that stores the CPT of a variable X given its parents: $P(X|Y)$. Each internal node is labeled with a variable $Y \in Y$ and has an outgoing branch for each possible state of that variable. Terminal nodes store probability distributions. To obtain the probability distribution $P(X|Y=y)$, the tree is traversed from the root, selecting at each internal node $Y \in Y$ the branch corresponding to the value of that variable in configuration y . This process leads to a terminal node labeled with the variable X , which represents the desired distribution. Notably, different configurations may lead to identical terminal nodes. In such cases, the tree can be simplified, potentially requiring less memory than storing the distribution as a CPT.

Conventional methods for constructing classification trees [26] can be applied to estimate a conditional probability distribution represented by a PT. In this work, we adopt a Bayesian approach, analogous to that used for learning Bayesian network structures in [10,27].

Algorithm 1 LearnTree(X, Y, D, S).

```

1:  $score(X, \emptyset) \leftarrow LBDE(X|\emptyset, D, S)$ 
2: // compute score for  $X$  parents
3: for each  $Y \in Y$  do
4:    $score(X, Y) \leftarrow LBDE(X|\{Y\}, D, S)$ 
5: end for
6:
7: //select  $Y \in Y$  with max. score
8:  $Y_m \leftarrow \operatorname{argmax}_{Y \in Y} score(X, Y)$ 
9:
10: // if the score improves when considering  $Y_m$  as a parent
11: if  $score(X, Y_m) > score(X, \emptyset)$  then
12:   // creates a new inner node for variable  $Y_m$ 
13:   creates  $T(Y_m)$ 
14:
15:   // loop on  $Y_m$  values
16:   for each  $y_t \in \Omega_{Y_m}$  do
17:     // select  $D$  cases where  $Y_m = y_t$ 
18:      $D' \leftarrow D(Y_m = y_t)$ 
19:
20:     // create node for  $y_t$  child
21:      $T(Y_m = y_t) \leftarrow \text{LearnTree}(X, Y \setminus \{Y_m\}, D', S/|\Omega_{Y_m}|)$ 
22:   end for
23: else
24:   // creates  $T$  as a leaf node for  $X$ 
25:   creates  $T(X)$ 
26:
27:   // iterates on  $X$  values to estimate  $p(X)$ 
28:   for each  $x_k \in \Omega_X$  do
29:     // count cases for  $X = x_k$ 
30:      $N_t \leftarrow |D(X = x_k)|$ 
31:
32:     // estimate probability for  $X = x$ 
33:      $p(X = x_k) = \frac{N_k + S/|\Omega_X|}{N + S}$ 
34:   end for
35:
36:   // sets computed distribution to  $T(X)$ 
37:    $T(X) = p(X)$ 
38: end if
```

The general outline of the algorithm for constructing the PT, named $\text{LearnTree}(X, pa(X), D, S)$, is presented in Alg. 1. This recursive algorithm is initially invoked with $Y = \emptyset$ as the parent set of X , and S as the global sample size. Subsequent recursive calls progressively modify the values of these arguments.

During execution, it may occur that no variable in Y satisfies the condition on line 11 of Algorithm 1. In such situations, the resulting trees can be highly simplified, as no internal nodes would be created for certain variables. To address this, the algorithm can be modified to incorporate a special case within the block beginning at line 23. Specifically, the procedure would identify a variable $Y_j \in Y$ for which there exists at least one other variable (for example, $Y_k \in Y$) such that

$$LBDE(X_i|\{Y_j, Y_k\}, D, S) > LBDE(X_i|\emptyset, D, S).$$

In this case, an internal node is created for the selected variable Y_j , and the algorithm proceeds with its standard execution.

To this end, the BDE score is employed to constrain the complexity of the PT by selectively incorporating only those variables that offer significant predictive power. When no individual variable satisfies the inclusion criteria, a look-ahead procedure is performed to evaluate whether pairs of variables are jointly informative. This mechanism effectively identifies higher-order dependencies where neither Y_1 nor

Y_2 is individually informative about X_i , yet their synergistic interaction provides substantial information.

The complexity of building a PT is analogous to that of building a classical decision tree, $O(Nn^2)$, where n is the number of variables in $\mathbf{Y}$ [28].

4. Methods based on logistic regression

4.1. Fundamentals of logistic regression

Logistic regression is a model for estimating the conditional probability distribution of a multinomial variable X , with $|\Omega_X| = r$, given a set of continuous variables $\mathbf{Z}$, where $|\mathbf{Z}| = n$. The procedure involves determining: (a) an $(r-1) \times n$ matrix $\mathbf{A}$, with a row for each of the first $r-1$ states of X ($x_1, \dots, x_{r-1}$), and a column for each variable $Z \in \mathbf{Z}$; and (b) a vector $\mathbf{C}$ of size $r-1$. Given $\mathbf{A}$ and $\mathbf{C}$, for a specific configuration $\mathbf{z}$, an $(r-1)$ -dimensional vector $\mathbf{B}$ is computed as $\mathbf{B} = \mathbf{A}\mathbf{z} + \mathbf{C}$. The elements b_i of $\mathbf{B}$ ($i = 1, \dots, r-1$) estimate the log-odds:

$$b_i = \log \left(\frac{P(X = x_i | \mathbf{Z} = \mathbf{z})}{P(X = x_r | \mathbf{Z} = \mathbf{z})} \right). \quad (4)$$

For a given row i of $\mathbf{A}$, corresponding to $X = x_i$, the configuration $\mathbf{z}$, and the i -th element of $\mathbf{C}$, the value b_i is computed as:

$$b_i = a_{i1}z_1 + \dots + a_{in}z_n + c_i.$$

Since the probability vector sums to 1, each conditional probability is obtained as:

$$P(X = x_i | \mathbf{Z} = \mathbf{z}) = \frac{e^{b_i}}{1 + \sum_{l=1}^{r-1} e^{b_l}}, \quad i = 1, \dots, r-1, \quad (5)$$

while

$$P(X = x_r | \mathbf{Z} = \mathbf{z}) = \frac{1}{1 + \sum_{l=1}^{r-1} e^{b_l}}. \quad (6)$$

Let F be the function that transforms $\mathbf{B}$ into probability estimates according to Eqs. (5) and (6). Its inverse, F^{-1} , computes a vector $\mathbf{B}$ from probability estimates using Eq. (4). Thus, the conditional probability for parameters $\mathbf{A}$ and $\mathbf{C}$ is

$$P_{(\mathbf{A}, \mathbf{C})}(X | \mathbf{Z} = \mathbf{z}) = F(\mathbf{A}\mathbf{z} + \mathbf{C}).$$

The elements of $\mathbf{A}$ and $\mathbf{C}$ are typically estimated via maximum likelihood using iterative methods such as Newton-Raphson [29]. Regularization is often applied; here, we employ $L1$ regularization by adding $\alpha(-\|\mathbf{A}\|_1 - \|\mathbf{C}\|_1)$ to the log-likelihood, where $\|\mathbf{A}\|_1$ represents the sum of absolute values of the elements of $\mathbf{A}$, and $\alpha > 0$. This promotes smaller parameter values, potentially driving some to zero. The larger α is, the stronger the regularization becomes, resulting in models with fewer non-zero parameters. This regularization is crucial because, as we propose the creation of new variables, $L1$ implicitly selects the most relevant ones, thereby mitigating the overfitting problem.

We have employed the SAGA (Stochastic Average Gradient Augmented) procedure for estimation [30], which has a complexity of $O(nN)$, because it is computationally efficient and supports $L1$ regularization.

4.2. Transformations of variables

When applying logistic regression to Bayesian networks, the parent variables $pa(X)$ are discrete, whereas the logistic regression model requires continuous inputs. The target distribution is $P(X|\mathbf{Y})$, where $pa(X) = \mathbf{Y}$. We define a deterministic transformation T to map $\mathbf{Y}$ into continuous variables $\mathbf{Z}$: $\mathbf{Z} = T(\mathbf{Y})$, with $T : \Omega_Y \rightarrow \Omega_Z$. The conditional probability is then computed as:

$$P_{(T, \mathbf{A}, \mathbf{C})}(X | \mathbf{Y} = \mathbf{y}) = P_{(\mathbf{A}, \mathbf{C})}(X | \mathbf{Z} = T(\mathbf{y})). \quad (7)$$

The transformation can be applied to a single configuration $\mathbf{y}$, yielding a vector $\mathbf{B}$ of parameters for that configuration, or to all configurations in Ω_Y ; then $T(\mathbf{Y})$ has dimension $(n \times L)$, where n is the number

of variables in $\mathbf{Z}$ and L is the number of configurations of $\mathbf{Y}$. In this case, the corresponding matrix of log-odds is:

$$\mathbf{B}(\mathbf{Z}) = \mathbf{A}T(\mathbf{Y}) + \mathbf{C}, \quad (8)$$

where $\mathbf{B}(\mathbf{Z})$ and $\mathbf{C}$ have dimensions $(r-1) \times L$, and each column is used to estimate probabilities for the corresponding configuration in Ω_Y .

4.2.1. Notation for transformations

We now establish several notational conventions for describing the forthcoming transformations and their properties:

- i index denotes the values of X ($i = 1, \dots, r$).
- j represents the index of variables in $pa(X) = \mathbf{Y}$ ($j = 1, \dots, |\mathbf{Y}|$).
- q indexes variables in $\mathbf{Z}$ ($q = 1, \dots, n$); sometimes composite indices clarify the mapping between $\mathbf{Z}$ and $\mathbf{Y}$.
- k refers to the possible values of Y_j ($k = 1, \dots, |\Omega_{Y_j}|$).
- m index represents a joint configuration of all variables in $\mathbf{Y}$ ($m = 1, \dots, L$).
- Continuous variables and associated parameters in transformations are distinguished to avoid ambiguity in examples.

4.2.2. Common settings for examples

To illustrate the transformations, we use a recurring example for learning the conditional distribution $P(X|\mathbf{Y})$, defined by the following features:

- X has two values, $\Omega_X = \{x_1, x_2\}$.
- $\mathbf{Y} = Y_1, Y_2$, each Y_j with $\Omega_{Y_j} = \{y_{j1}, y_{j2}, y_{j3}\}$.
- The total number of configurations of $\mathbf{Y}$ is $L = 9$, ordered as:

$$\Omega_Y = \{(Y_1 = y_{11}, Y_2 = y_{21}), \dots, (Y_1 = y_{13}, Y_2 = y_{23})\}$$

We now examine basic transformations and their properties, starting with a result that simplifies model parameterization. **Proposition 1** shows that setting $\mathbf{C} = 0$ imposes no restriction, as any conditional probability distribution estimable via T can also be estimated using a simpler transformation T_s with $(\mathbf{A}_s, \mathbf{C}_s = 0)$.

4.3. Basic transformations and properties

Proposition 1. Let T be a transformation and T_s be derived by adding a constant variable $Z_c = 1$. For any $(\mathbf{A}, \mathbf{C})$ associated with T , there exists $(\mathbf{A}_s, 0)$ for T_s with $\mathbf{A}_s = (\mathbf{A} \ \mathbf{C})$ such that for any $\mathbf{y}$:

$$\mathbf{A}T(\mathbf{y}) + \mathbf{C} = \mathbf{A}_s T_s(\mathbf{y})$$

The proof is on [Appendix A](#).

Henceforth, we assume $\mathbf{C} = 0$ and omit it from Eq. (7).

4.3.1. Naive transformation

The first transformation we consider is the naive transformation, represented by T_n (n - naive). This approach, which depends on a pre-defined ordering of the parent variables' values, introduces a continuous variable Z_j for each parent $Y_j \in \mathbf{Y}$. The value of Z_j is set to $k-1$ when Y_j takes its k -th value, y_{jk} (assuming k starts at 1).

For instance, if Y_j has the ordered values $\{y_{j1}, y_{j2}, y_{j3}\}$, the corresponding variable Z_j in $\mathbf{Z}$ will take values from $\{0, 1, 2\}$. Here, $Z_j = 0$ represents the case $Y_j = y_{j1}$, $Z_j = 1$ corresponds to $Y_j = y_{j2}$, and $Z_j = 2$ signifies $Y_j = y_{j3}$. Additionally, this transformation introduces a constant variable Z_c that always takes the value 1.

The matrix $\mathbf{A}_n$ contains one parameter for each pair of X values and $\mathbf{Z}$ variables, (x_i, Z_j) . The parameters in $\mathbf{A}_n$ are written as a_{ij} , where i relates to the state of X and j to the associated variable in $\mathbf{Z}$. For the constant variable Z_c , the parameters are labeled a_{ic} . The matrix $\mathbf{A}_n$ has dimensions $((r-1) \times |\mathbf{Z}|)$, with a row for each of the first $r-1$ states of X and a column for each variable in $\mathbf{Z}$.

Example 1. Using the common setting from Section 4.2.2 and assuming the default value ordering, Y_1 is transformed into Z_1 with $\Omega_{Z_1} = \{0, 1, 2\}$, and Y_2 into Z_2 with $\Omega_{Z_2} = \{0, 1, 2\}$. Thus, $\mathbf{Z} = \{Z_1, Z_2, Z_c\}$, which means $n = 3$. The number of configurations for $\mathbf{Z}$ is $|\Omega_{\mathbf{Z}}| = |\Omega_Y| = 9 = L$.

The matrix $\mathbf{A}_n$ has dimensions 1×3 (one row for $X = x_1$), with a column for each variable in $\mathbf{Z}$. In other words, it contains the following parameters:

$$\mathbf{A}_n = \begin{pmatrix} \alpha_{11} & \alpha_{12} & \alpha_{1c} \end{pmatrix}$$

We can now calculate $\mathbf{B}_n(\mathbf{Z})$ for any configuration of $\mathbf{Y}$ using $\mathbf{B}_n(\mathbf{Z}) = \mathbf{A}_n T_n(\mathbf{Y})$, where $T_n(\mathbf{Y})$ is a 3×9 matrix:

$$T_n(\mathbf{Y}) = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 2 & 2 & 2 \\ 0 & 1 & 2 & 0 & 1 & 2 & 0 & 1 & 2 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{matrix} Z_1 \\ Z_2 \\ Z_c \end{matrix} \quad (9)$$

$Y_1 \quad Y_2 \quad Y_3 \quad Y_4 \quad Y_5 \quad Y_6 \quad Y_7 \quad Y_8 \quad Y_9$

As shown, the columns of $T_n(\mathbf{Y})$ contain the transformed versions of all possible configurations of $\mathbf{Y}$. For instance, the first column represents the transformation of $\mathbf{y} = (Y_1 = y_{11}, Y_2 = y_{21})$ into $\mathbf{z} = (Z_1 = 0, Z_2 = 0, Z_c = 1)$. The rows correspond to the values of Z_1 , Z_2 , and the constant $Z_c = 1$, respectively. The full computation of $\mathbf{B}_n(\mathbf{Z})$ is shown in Eq. (10):

$$\mathbf{B}_n(\mathbf{Z}) = \mathbf{A}_n T_n(\mathbf{Y}) = \begin{pmatrix} \alpha_{11} & \alpha_{12} & \alpha_{1c} \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 2 & 2 & 2 \\ 0 & 1 & 2 & 0 & 1 & 2 & 0 & 1 & 2 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad (10)$$

The resulting values from Eq. (10) are presented below (the transposed vector $\mathbf{B}_n^T(\mathbf{Z})$ is shown for better visualization):

$$\mathbf{B}_n^T(\mathbf{Z}) = \begin{pmatrix} \alpha_{1c} \\ \alpha_{12} + \alpha_{1c} \\ 2\alpha_{12} + \alpha_{1c} \\ \alpha_{11} + \alpha_{1c} \\ \alpha_{11} + \alpha_{12} + \alpha_{1c} \\ \alpha_{11} + 2\alpha_{12} + \alpha_{1c} \\ 2\alpha_{11} + \alpha_{1c} \\ 2\alpha_{11} + \alpha_{12} + \alpha_{1c} \\ 2\alpha_{11} + 2\alpha_{12} + \alpha_{1c} \end{pmatrix} \quad (11)$$

And according to Eq. (6):

$$P_{(T_n, \mathbf{A}_n)}(X = b | Y_1 = a, Y_2 = a) = P_{(T_n, \mathbf{A}_n)}(X = b | \mathbf{Z} = \{0, 0, 1\}) = \frac{1}{1 + e^{\alpha_{1c}}}$$

$$P_{(T_n, \mathbf{A}_n)}(X = b | Y_1 = a, Y_2 = b) = P_{(T_n, \mathbf{A}_n)}(X = b | \mathbf{Z} = \{0, 1, 1\}) = \frac{1}{1 + e^{\alpha_{12} + \alpha_{1c}}}$$

.....

$$P_{(T_n, \mathbf{A}_n)}(X = b | Y_1 = c, Y_2 = c) = P_{(T_n, \mathbf{A}_n)}(X = b | \mathbf{Z} = \{2, 2, 1\}) = \frac{1}{1 + e^{2\alpha_{11} + 2\alpha_{12} + \alpha_{1c}}}$$

F^{-1} values are:

$$F^{-1}(P_{(T_n, \mathbf{A}_n)}(X = b | Y_1 = a, Y_2 = a)) = \alpha_{1c}$$

$$F^{-1}(P_{(T_n, \mathbf{A}_n)}(X = b | Y_1 = a, Y_2 = b)) = \alpha_{12} + \alpha_{1c}$$

.....

$$F^{-1}(P_{(T_n, \mathbf{A}_n)}(X = b | Y_1 = c, Y_2 = c)) = 2\alpha_{11} + 2\alpha_{12} + \alpha_{1c}$$

4.3.2. Dummy transformation

The second transformation, based on *dummy* variables, is represented by T_d (d - *dummy*). Each parent $Y_j \in \mathbf{Y}$ is mapped to a set of *dummy* variables. If the domain of Y_j is $\{y_{j1}, \dots, y_{jk}\}$, this variable is transformed into $k - 1$ new variables ($Z_{j2}, \dots, Z_{jk}$) which are added to $\mathbf{Z}$. In the notation Z_{jk} , the first subscript j corresponds to the parent variable Y_j , and the second subscript k corresponds to the value y_{jk} . Each variable Z_{jk} takes the value 1 only when $Y_j = y_{jk}$ and 0 otherwise. Note that no variable is created for the first state, y_{j1} ; thus, the index k for the *dummy* variables runs from 2 to $|\Omega_{Y_j}|$. This transformation also requires adding the constant variable Z_c .

The parameters in the matrix $\mathbf{A}_d$ are written as β_{ijk} , where i relates to the state of X , j to the parent variable Y_j , and k to the specific state of that parent. For the constant variable Z_c , the corresponding parameters are labeled β_{ic} . The matrix $\mathbf{A}_d$ has dimension $((r-1) \times |\mathbf{Z}|)$. The number of new variables is $|\mathbf{Z}| = \sum_{Y_j \in \mathbf{Y}} (|\Omega_{Y_j}| - 1)$. We can observe that this number of variables grows linearly with the number of parents, in contrast to the number of parameters of a probability table, which increases exponentially. The final complexity of applying logistic regression is $O(nN)$, where n represents the number of these variables. Therefore, it can be stated that this procedure scales effectively with the number of parents.

Example 2. Following the common setting in Section 4.2.2, five variables are needed: four *dummy* variables for the second and third values of each variable in $\mathbf{Y}$, as well as the constant variable Z_c . The resulting set of variables is: $\mathbf{Z} = \{Z_{12}, Z_{13}, Z_{22}, Z_{23}, Z_c\}$ (so $n = 5$). The matrix $\mathbf{A}_d$ has dimension (1×5) and contains the following parameters:

$$\mathbf{A}_d = \begin{pmatrix} \beta_{112} & \beta_{113} & \beta_{122} & \beta_{123} & \beta_{1c} \end{pmatrix}$$

$\mathbf{B}_d(\mathbf{Z}) = \mathbf{A}_d T_d(\mathbf{Y})$ is computed as follows:

$$\mathbf{B}_d(\mathbf{Z}) = \mathbf{A}_d T_d(\mathbf{Z}) = \begin{pmatrix} \beta_{112} & \beta_{113} & \beta_{122} & \beta_{123} & \beta_{1c} \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \quad (12)$$

The transpose of the result from Eq. (12) is shown below for better visualization:

$$\mathbf{B}_d^T(\mathbf{Z}) = \begin{pmatrix} \beta_{1c} \\ \beta_{122} + \beta_{1c} \\ \beta_{123} + \beta_{1c} \\ \beta_{112} + \beta_{1c} \\ \beta_{112} + \beta_{122} + \beta_{1c} \\ \beta_{112} + \beta_{123} + \beta_{1c} \\ \beta_{113} + \beta_{1c} \\ \beta_{113} + \beta_{122} + \beta_{1c} \\ \beta_{113} + \beta_{123} + \beta_{1c} \end{pmatrix} \quad (13)$$

According to Eq. (6):

$$P_{(T_d, \mathbf{A}_d)}(X = b | Y_1 = a, Y_2 = a) = P_{(T_d, \mathbf{A}_d)}(X = b | \mathbf{Z} = \{0, 0, 0, 0, 1\}) = \frac{1}{1 + e^{\beta_{1c}}}$$

$$P_{(T_d, \mathbf{A}_d)}(X = b | Y_1 = a, Y_2 = b) = P_{(T_d, \mathbf{A}_d)}(X = b | \mathbf{Z} = \{0, 0, 1, 0, 1\}) = \frac{1}{1 + e^{\beta_{122} + \beta_{1c}}}$$

.....

$$P_{(T_d, \mathbf{A}_d)}(X = b | Y_1 = c, Y_2 = c) = P_{(T_d, \mathbf{A}_d)}(X = b | \mathbf{Z} = \{0, 1, 0, 1, 1\}) = \frac{1}{1 + e^{\beta_{113} + \beta_{123} + \beta_{1c}}}$$

T_n is constrained by parameter dependencies and cannot generate arbitrary probability distributions. For example, attempting to model the following distribution is impossible with T_n :

$$P_{(T_n, \mathbf{A}_n)}(X = b \mid Y_1 = a) = 0.2$$

$$P_{(T_n, \mathbf{A}_n)}(X = b \mid Y_1 = b) = 0.6$$

$$P_{(T_n, \mathbf{A}_n)}(X = b \mid Y_1 = c) = 0.4$$

To obtain $P(X = b \mid Y_1 = a) = 0.2$, the parameter α_{1c} must be 1.386. To then obtain $P(X = b \mid Y_1 = b) = 0.6$, α_{11} must be 0.667. These fixed parameters then determine the third probability, yielding $P(X = b \mid Y_1 = c) = 0.152$, which conflicts with the target value of 0.4. $\triangleleft$

4.3.5. Aggregation of linear combinations of variables

Proposition 2 is useful for establishing relationships between transformations.

Proposition 2. Let T_1 and T_2 be two transformations such that $T_1 \leq T_2$. Suppose a new transformation T_{1n} is created by adding a variable to T_1 that is a linear combination of the variables in T_2 . It then follows that $T_{1n} \leq T_2$.

The proof is on [Appendix A](#).

4.3.6. Rijmen transformation

Rijmen [7] proposed a transformation, which we represent as T_c (c - configuration). This transformation defines one variable Z_j for each configuration y_j of the parent set $pa(X) = \mathbf{Y}$. Recall that $L = |\Omega_{\mathbf{Y}}|$ is the number of parent configurations. We can therefore index these configurations as $\Omega_{\mathbf{Y}} = \{y_1, \dots, y_L\}$. The variable Z_j , associated with the configuration y_j , takes the value 1 if that configuration occurs and 0 otherwise. The constant variable Z_c is not needed in this case. The set of new variables is thus $\mathbf{Z} = \{Z_1, \dots, Z_L\}$. Here, there is a one-to-one correspondence between configurations and variables, so the terms can be used interchangeably.

The parameters of $\mathbf{A}_c$ are written as γ_{im} , where i corresponds to the state of X and m to the variable/configuration index. The matrix $\mathbf{A}_c$ has dimensions $(r-1) \times L$, with rows corresponding to the states of X (except the last) and columns corresponding to the variables/configurations.

Example 5. Using the common setting from the previous examples, the set $\mathbf{Z}$ will contain nine variables, one for each configuration of $\mathbf{Y} = \{Y_1, Y_2\}$ (both with three states). For example, $Z_1 = 1$ only for the configuration $y_1 = (Y_1 = a, Y_2 = a)$ and is 0 otherwise; $Z_2 = 1$ only for $y_2 = (Y_1 = a, Y_2 = b)$ and is 0 otherwise, and so on. Therefore, the matrix $\mathbf{A}_c$ will have one row and nine columns, $\mathbf{A}_c = (\gamma_{11} \dots \gamma_{19})$. Given the one-to-one mapping, the transformation matrix $T_c(\mathbf{Y})$ is the 9×9 identity matrix, $\mathbf{I}_9$. As in the previous examples, we assume $\mathbf{C} = \mathbf{0}$. For this transformation, the computation of $\mathbf{B}_c(\mathbf{Z})$ simplifies to $\mathbf{A}_c T_c(\mathbf{Y}) = \mathbf{A}_c \mathbf{I}_9 = \mathbf{A}_c$:

$$\begin{aligned} \mathbf{A}_c T_c(\mathbf{Y}) &= (\gamma_{11} \quad \gamma_{12} \quad \dots \quad \gamma_{19}) \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \\ &= (\gamma_{11} \quad \gamma_{12} \quad \dots \quad \gamma_{19}) \end{aligned} \quad (14)$$

Applying [Eq. \(6\)](#), we obtain the following probabilities:

$$\begin{aligned} P_{(T_c, \mathbf{A}_c)}(X = b \mid Y_1 = a, Y_2 = a) &= P_{(T_c, \mathbf{A}_c)}(X = b \mid \mathbf{Z} = \{1, 0, 0, 0, 0, 0, 0, 0, 0\}) \\ &= \frac{1}{1 + e^{\gamma_{11}}} \end{aligned}$$

$$\begin{aligned} P_{(T_c, \mathbf{A}_c)}(X = b \mid Y_1 = a, Y_2 = b) &= P_{(T_c, \mathbf{A}_c)}(X = b \mid \mathbf{Z} = \{0, 1, 0, 0, 0, 0, 0, 0, 0\}) \\ &= \frac{1}{1 + e^{\gamma_{12}}} \end{aligned}$$

.....

$$\begin{aligned} P_{(T_c, \mathbf{A}_c)}(X = b \mid Y_1 = c, Y_2 = c) &= P_{(T_c, \mathbf{A}_c)}(X = b \mid \mathbf{Z} = \{0, 0, 0, 0, 0, 0, 0, 0, 1\}) \\ &= \frac{1}{1 + e^{\gamma_{19}}} \end{aligned}$$

The corresponding F^{-1} values are:

$$F^{-1}(P_{(T_c, \mathbf{A}_c)}(X = b \mid Y_1 = a, Y_2 = a)) = \gamma_{11}$$

$$F^{-1}(P_{(T_c, \mathbf{A}_c)}(X = b \mid Y_1 = a, Y_2 = b)) = \gamma_{12}$$

.....

$$F^{-1}(P_{(T_c, \mathbf{A}_c)}(X = b \mid Y_1 = c, Y_2 = c)) = \gamma_{19} \quad \triangleleft$$

The primary drawback of this transformation is its requirement of L variables, a number that grows exponentially with the size of the parent set.

4.3.7. T_{ds} : transformation derived from T_d using subsets of variables

The T_{ds} transformation (d - dummy, s - subset) is derived from T_d by considering subsets of dummy variables. It was proposed by Rijmen [7] to alleviate the scalability issues of T_c . In this transformation, the new variables are products of subsets of dummy variables. For this reason, we use the notation $\mathbf{H}_j$ instead of Z_j for these new variables. Given a subset $\mathbf{H}_j = \{Z_1, \dots, Z_k\}$, its value is computed as their product: $\mathbf{H}_j = \prod_{Z_i \in \mathbf{H}_j} Z_i$. This transformation also includes the constant variable Z_c (associated with $\mathbf{H}_1 = \{Z_c\}$). Crucially, these subsets are formed with the restriction that no subset can contain two variables corresponding to different values of the same parent variable Y_j . The total set of new variables $\mathbf{H}_j$ is denoted by $\mathbf{H}$. Since the underlying dummy variables are binary, the variables in $\mathbf{H}$ are also binary.

Let D be the number of dummy variables used in T_d . As mentioned before, the first subset is $\mathbf{H}_1 = \{Z_c\}$. The next D subsets are $\mathbf{H}_2, \dots, \mathbf{H}_{D+1}$, each containing a single dummy variable. Subsequent subsets are formed from pairs of variables (subject to the restriction mentioned above), triplets, and so on, up to subsets containing $|\mathbf{Y}|$ variables. Typically, the set $\mathbf{H}$ is computed by considering all subsets $\mathbf{H}_j$ up to a cardinality M . In this way, $M = 1$ yields the T_d transformation (dummy variables only), while $M = 2$ incorporates pairwise interactions, and so forth. The overall complexity is determined by the value of M .

The subsets are indexed by j , where $j = 1, \dots, |\mathbf{H}|$. The matrix $\mathbf{A}_{ds}$ has dimension $((r-1) \times |\mathbf{H}|)$. Its parameters are written as δ_{ij} , where the first subscript corresponds to the state of X and the second to the subset $\mathbf{H}_j$. The following example describes the application of T_{ds} to the common setting from [Section 4.2.2](#).

Example 6. As established in [Example 2](#), T_d requires five variables: the constant Z_c and four dummy variables ($D = 4$). That is, $\mathbf{Z} = \{Z_{12}, Z_{13}, Z_{22}, Z_{23}, Z_c\}$. Therefore, the subsets used as variables will be:

$$\mathbf{H}_1 = \{Z_c\}, \mathbf{H}_2 = \{Z_{12}\}, \mathbf{H}_3 = \{Z_{13}\}, \mathbf{H}_4 = \{Z_{22}\}, \mathbf{H}_5 = \{Z_{23}\}$$

$$\mathbf{H}_6 = \{Z_{12}, Z_{22}\}, \mathbf{H}_7 = \{Z_{12}, Z_{23}\}, \mathbf{H}_8 = \{Z_{13}, Z_{22}\}, \mathbf{H}_9 = \{Z_{13}, Z_{23}\}$$

The matrix $\mathbf{A}_{ds}$ has dimensions (1×9) (since $|\mathbf{H}| = 9$) and contains the parameters indicated below:

$$\mathbf{A}_{ds} = (\delta_{11} \quad \delta_{12} \quad \dots \quad \delta_{18} \quad \delta_{19})$$

$\mathbf{B}_{ds}(\mathbf{Z}) = \mathbf{A}_{ds}T_{ds}(\mathbf{Y})$ is computed as follows:

$$\mathbf{B}_{ds}(\mathbf{Z}) = \mathbf{A}_{ds}T_{ds}(\mathbf{Y}) = (\delta_{11} \ \delta_{12} \ \dots \ \delta_{18} \ \delta_{19}) \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \quad (15)$$

The transpose of the resulting vector, $\mathbf{B}_{ds}^T(\mathbf{Z})$, is:

$$\mathbf{B}_{ds}^T(\mathbf{Z}) = \begin{pmatrix} \delta_{11} \\ \delta_{11} + \delta_{14} \\ \delta_{11} + \delta_{15} \\ \delta_{11} + \delta_{12} \\ \delta_{11} + \delta_{12} + \delta_{14} + \delta_{16} \\ \delta_{11} + \delta_{12} + \delta_{15} + \delta_{17} \\ \delta_{11} + \delta_{13} \\ \delta_{11} + \delta_{13} + \delta_{14} + \delta_{18} \\ \delta_{11} + \delta_{13} + \delta_{15} + \delta_{19} \end{pmatrix} \quad (16)$$

According to Eq. (6):

$$\begin{aligned} P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|Y_1 = a, Y_2 = a) &= P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|\mathbf{H}) \\ &= \{1, 0, 0, 0, 0, 0, 0, 0, 0\} = \frac{1}{1 + e^{\delta_{11}}} \\ P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|Y_1 = a, Y_2 = b) &= P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|\mathbf{H}) \\ &= \{1, 0, 0, 1, 0, 0, 0, 0, 0\} = \frac{1}{1 + e^{\delta_{11} + \delta_{14}}} \\ &\dots \dots \dots \\ P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|Y_1 = c, Y_2 = c) &= P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|\mathbf{H}) \\ &= \{1, 0, 1, 0, 1, 0, 0, 0, 1\} \\ &= \frac{1}{1 + e^{\delta_{11} + \delta_{13} + \delta_{15} + \delta_{19}}} \end{aligned}$$

The corresponding F^{-1} values are:

$$\begin{aligned} F^{-1}(P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|Y_1 = a, Y_2 = a)) &= \delta_{11} \\ F^{-1}(P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|Y_1 = a, Y_2 = b)) &= \delta_{11} + \delta_{14} \\ &\dots \dots \dots \\ F^{-1}(P_{(T_{ds}, \mathbf{A}_{ds})}(X = b|Y_1 = c, Y_2 = c)) &= \delta_{11} + \delta_{13} + \delta_{15} + \delta_{19} \end{aligned} \quad \triangleleft$$

Both transformations T_c and T_{ds} are always complete and equivalent, as the following result shows.

4.3.8. Equivalence of T_c and T_{ds}

Proposition 3. If X is a variable and $\mathbf{Y}$ is its set of parents, then T_c and T_{ds} are equivalent, $T_c \equiv T_{ds}$.

The proof is on [Appendix A](#).

As a practical example, consider the configuration from [Examples 5 and 6](#). Specifically, for the configuration $\mathbf{y}_6 = (Y_1 = y_{12}, Y_2 = y_{23})$, we must equate the sixth component of the resulting vectors from [Eqs. 14 and \(16\)](#). This yields:

$$\gamma_{16} = \delta_{11} + \delta_{12} + \delta_{15} + \delta_{17}.$$

4.3.9. $T_{d\oplus}$: transformation derived from T_d and using $\oplus$ for subsets of variables

Another transformation derived from T_d uses the XOR operation ($\oplus$) instead of multiplication. This transformation, represented by $T_{d\oplus}$ (d -

dummy, $\oplus$ - XOR), is also complete. This approach aims to mitigate an issue with T_{ds} : variables in $\mathbf{H}_j$ representing products of multiple binary variables tend to be sparse, often resulting in near-zero values that contribute negligible information to the model. This problem is particularly acute when the number of variables in the subset $\mathbf{H}_j$ is large. To avoid this, this alternative procedure constructs new variables using the XOR operator. The XOR of two binary variables Z_1 and Z_2 is defined as:

$$Z_1 \oplus Z_2 = Z_1 + Z_2 - 2Z_1Z_2 \quad (17)$$

Extending this operation to a third variable, Z_3 , yields:

$$\begin{aligned} (Z_1 \oplus Z_2) \oplus Z_3 &= (Z_1 \oplus Z_2) + Z_3 - 2(Z_1 \oplus Z_2)Z_3 \\ &= Z_1 + Z_2 + Z_3 - 2Z_1Z_2 - 2Z_1Z_3 - 2Z_2Z_3 \\ &\quad + 4Z_1Z_2Z_3 \end{aligned} \quad (18)$$

Since the XOR operation is associative, it can be extended to any set of variables $\mathbf{H}$ as $\bigoplus_{Z_j \in \mathbf{H}} Z_j$. We will consider that $\bigoplus_{Z_j \in \mathbf{H}} Z_j = \emptyset = Z_c$. The dimensionality of $\mathbf{A}_{d\oplus}$ is identical to that of $\mathbf{A}_{ds}$: $((r-1) \times |\mathbf{H}|)$. The parameters are written as ϵ_{ij} , where i indicates that $X = x_i$ and j refers to the subset $\mathbf{H}_j$.

Example 7. As in [Example 2](#), in T_d , $\mathbf{Z} = \{Z_{12}, Z_{13}, Z_{22}, Z_{23}, Z_c\}$. The subsets for $T_{d\oplus}$ are the same as those used for T_{ds} :

$$\begin{aligned} H_1 &= \emptyset, H_2 = \{Z_{12}\}, H_3 = \{Z_{13}\}, H_4 = \{Z_{22}\}, H_5 = \{Z_{23}\} \\ H_6 &= \{Z_{12}, Z_{22}\}, H_7 = \{Z_{12}, Z_{23}\}, H_8 = \{Z_{13}, Z_{22}\}, H_9 = \{Z_{13}, Z_{23}\} \end{aligned}$$

As with T_{ds} , the matrix $\mathbf{A}_{d\oplus}$ has dimensions (1×9) (since $|\mathbf{H}| = 9$) and contains the following parameters:

$$\mathbf{A}_{ds} = \begin{pmatrix} \epsilon_{11} & \epsilon_{12} & \dots & \epsilon_{18} & \epsilon_{19} \end{pmatrix}$$

The computation of $\mathbf{B}_{d\oplus}(\mathbf{Z}) = \mathbf{A}_{d\oplus}T_{d\oplus}(\mathbf{Y})$ involves applying the XOR operation to the variables within each subset:

$$\begin{aligned} \mathbf{B}_{d\oplus}(\mathbf{Y}) &= \mathbf{A}_{d\oplus}T_{d\oplus}(\mathbf{Y}) \\ &= (\epsilon_{11} \ \epsilon_{1,2} \ \dots \ \epsilon_{18} \ \epsilon_{19}) \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix} \end{aligned} \quad (19)$$

The transpose of the resulting vector, $\mathbf{B}_{d\oplus}^T(\mathbf{Y})$, is:

$$\mathbf{B}_{d\oplus}^T(\mathbf{Y}) = \begin{pmatrix} \epsilon_{11} \\ \epsilon_{11} + \epsilon_{14} + \epsilon_{16} + \epsilon_{18} \\ \epsilon_{11} + \epsilon_{15} + \epsilon_{17} + \epsilon_{19} \\ \epsilon_{11} + \epsilon_{12} + \epsilon_{16} + \epsilon_{17} \\ \epsilon_{11} + \epsilon_{12} + \epsilon_{14} + \epsilon_{17} + \epsilon_{18} \\ \epsilon_{11} + \epsilon_{12} + \delta_{15} + \epsilon_{16} + \epsilon_{19} \\ \epsilon_{11} + \epsilon_{13} + \epsilon_{18} + \epsilon_{19} \\ \epsilon_{11} + \epsilon_{13} + \epsilon_{14} + \epsilon_{15} + \epsilon_{19} \\ \epsilon_{11} + \epsilon_{13} + \epsilon_{15} + \epsilon_{17} + \epsilon_{18} \end{pmatrix} \quad (20)$$

According to [Eq. \(6\)](#):

$$\begin{aligned} P_{(T_{d\oplus}, \mathbf{A}_{d\oplus})}(X = b|Y_1 = a, Y_2 = a) &= P_{(T_{d\oplus}, \mathbf{A}_{d\oplus})}(X = b|\mathbf{H}) \\ &= \{1, 0, 0, 0, 0, 0, 0, 0, 0\} = \frac{1}{1 + e^{\epsilon_{11}}} \end{aligned}$$

$$P_{(T_{d\oplus}, A_{d\oplus})}(X = b|Y_1 = a, Y_2 = b) = P_{(T_{d\oplus}, A_{d\oplus})}(X = b|\mathbf{H})$$

$$= \{1, 0, 0, 1, 0, 1, 0, 1, 0\}$$

$$= \frac{1}{1 + e^{\epsilon_{11} + \epsilon_{14} + \epsilon_{16} + \epsilon_{18}}}$$

$$P_{(T_{d\oplus}, A_{d\oplus})}(X = b|Y_1 = c, Y_2 = c) = P_{(T_{d\oplus}, A_{d\oplus})}(X = b|\mathbf{H})$$

$$= \{1, 0, 1, 0, 1, 0, 1, 1, 0\}$$

$$= \frac{1}{1 + e^{\epsilon_{11} + \epsilon_{13} + \epsilon_{15} + \epsilon_{17} + \epsilon_{18}}}$$

The corresponding F^{-1} values are:

$$F^{-1}(P_{(T_{d\oplus}, A_{d\oplus})}(X = b|Y_1 = a, Y_2 = a)) = \epsilon_{11}$$

$$F^{-1}(P_{(T_{d\oplus}, A_{d\oplus})}(X = b|Y_1 = a, Y_2 = b)) = \epsilon_{11} + \epsilon_{14} + \epsilon_{16} + \epsilon_{18}$$

$$F^{-1}(P_{(T_{d\oplus}, A_{d\oplus})}(X = b|Y_1 = c, Y_2 = c)) = \epsilon_{11} + \epsilon_{13} + \epsilon_{15} + \epsilon_{17} + \epsilon_{18} \quad \triangleleft$$

4.3.10. *Xor as product of variables*

Proposition 4. Let T_1 and T_2 be two equivalent transformations, $T_1 \equiv T_2$, and let $\mathbf{H}$ be a subset of binary variables common to both transformations. If for any subset $\mathbf{H}_s \subseteq \mathbf{H}$ the variable $\prod_{Z_j \in \mathbf{H}_s} Z_j$ is added to T_1 and the variable produced as a result of $\bigoplus_{Z_j \in \mathbf{H}_s} Z_j$ is added to T_2 , thus obtaining two new transformations T_{1n} and T_{2n} , then it holds that $T_{1n} \equiv T_{2n}$.

The proof is on [Appendix A](#).

4.3.11. $T_{d\oplus r}$: limiting the number of variables

Using a complete transformation is not always desirable because its ability to model any CPT means that the number of parameters cannot be reduced. To simplify the model, a restricted version of T_{ds} that limits the size of the subsets used was introduced in [7]. This work proposes a further refinement: building the set of subsets greedily. Starting with single variables, larger subsets are formed only if they provide significantly more information than their constituent parts. This restricted transformation is represented by $T_{d\oplus r}$ (d - dummy, $\oplus$ - XOR, r - restricted). The selection procedure is detailed in [Algorithm 2](#). The algorithm requires the following inputs:

- The set of *dummy* variables from T_d .
- M : the maximum size of the subsets to be considered.
- D : the learning dataset.
- ϵ : a threshold for score comparisons.
- f : the scoring function to use (e.g., *BIC*, *BDE*).

The result will be the set of selected subsets, $\mathbf{H}$, for this transformation.

The condition that Z_k is compatible with H_i (line 19) ensures that Z_k does not correspond to the same parent variable Y_p as any other *dummy* variable already in H_i . This prevents combining variables that represent mutually exclusive states of the same parent.

We used the *BIC* score as f , but the *BDE* score could also be used. The choice of the scoring function is not critical here, as we consistently compare parent sets of the same dimension (a single binary variable). We selected *BIC* for its computational simplicity. ϵ is always a small value; its role is to ensure that an added variable U provides significantly more information than either of the two combined variables. This prevents the addition of variables whose marginal information gain is merely due to rounding errors. M serves the same purpose as in T_{ds} , limiting the complexity and the number of parameters. For example, with $M = 2$, only pairwise interactions are considered, and the number of added variables is bounded by a quadratic function of the number of parents.

The main differences between this approach and Rijmen [7] can be summarized in the following points:

Algorithm 2 Variable selector($\mathbf{Z} \in T_d, M, D, \epsilon, f$).

```

1: // add constant variable to  $\mathbf{Z}$  to  $\mathbf{H}_1$ 
2: add variable  $Z_c$  to  $T_{d\oplus r}$  as  $\mathbf{H}_1$ 
3:
4: // add all single dummy variables
5: for any variable  $Z_j$  in  $T_d$  do
6:   add  $Z_j$  to  $T_{d\oplus r}$  as its associated set  $\mathbf{H}_{j+1} = \{Z_j\}$ 
7: end for
8:
9: // build and add larger subsets up to size  $M$ 
10: for  $j = 2$  to  $M$  do
11:
12:   // consider variables in  $T_d$ 
13:   for any variable  $Z_k$  in  $T_d$  do
14:
15:     // considers subsets with  $j - 1$  variables
16:     for any variable  $Z_l$  included in  $\mathbf{H}_l$  such that  $|\mathbf{H}_l| = j - 1$  do
17:
18:       // checks if  $Z_k$  is related to variables in  $\mathbf{Y}$  and included
19:       in  $\mathbf{H}_l$ 
20:       if  $Z_k$  is compatible with  $\mathbf{H}_l$  then
21:
22:         // compute the value for the new candidate variable
23:         compute  $U = Z_k \oplus Z_l$ 
24:
25:         // check the information gain due to  $U$ 
26:         if  $f(X, \{U\}, D) > \max(f(X, \{Z_k\}, D), f(X, \{Z_l\}, D)) +$ 
27:            $\epsilon$  then
28:
29:           // add the new variable to  $\mathbf{H}$ 
30:           add  $U$  to set  $\mathbf{H}_{l+1} = \mathbf{H}_l \cup \{Z_k\}$ 
31:         end if
32:       end if
33:     end for
34:   end for
35: end for

```

- The use of the XOR operation instead of the product does not constitute an essential difference, as the two models are theoretically equivalent. However, we observe that XOR produces more balanced sample sizes.
- In Rijmen's approach, there is no specific procedure for learning; this value is predetermined. In contrast, our approach utilizes the BIC score to determine which interactions are ultimately included. This renders our method more flexible and data-adaptive; however, it lacks an inherent limit on model size, which could theoretically result in an exponential number of parameters relative to the number of parents. Furthermore, it increases complexity, as scores must be computed each time a new XOR operation is analyzed in Step 25 of [Algorithm 2](#). In practice, we have observed that our approach adds only a small number of new variables, typically maintaining low model complexity.
- Rijmen's approach considers the case of missing data using the EM algorithm [31].
- Rijmen adds a small constant to all frequencies to avoid zero counts ($0.01N$). We excluded from our experiments any cases where a specific value of variable X_i was not observed in the training data, as the implementations utilized would otherwise produce an error.

4.4. Transformations based on linear discriminant analysis

Linear Discriminant Analysis (*LDA*) [13] considers a categorical variable X and a set of continuous predictor variables $\mathbf{Z}$. The goal is to find a $(n \times m)(n = |\mathbf{Z}|)$ coefficient matrix $\mathbf{W}$ that defines a set of m linear transformations. These transformations, given by $\mathbf{W}^T \mathbf{z}$, are chosen to

maximize the class separability for X , where $\mathbf{W}^T$ is the transpose of $\mathbf{W}$. Specifically, the transformation maximizes the ratio:

$$\frac{|\mathbf{W}^T \mathbf{S}_b \mathbf{W}|}{|\mathbf{W}^T \mathbf{S}_w \mathbf{W}|}, \quad (21)$$

where $|\cdot|$ represents the determinant, $\mathbf{S}_b$ is the between-class scatter matrix:

$$\mathbf{S}_b = \sum_{k=1}^{k_i} N_k (\mu_k - \mu)(\mu_k - \mu)^T,$$

and $\mathbf{S}_w$ is the within-class scatter matrix:

$$\mathbf{S}_w = \sum_l (\mathbf{y}_l - \mu_k)(\mathbf{y}_l - \mu_k)^T.$$

Here, N_k is the number of samples where $X = x_k$; μ_k is the mean vector of the variables in $\mathbf{Z}$ for samples in class x_k ; μ is the overall mean vector of $\mathbf{Z}$; and $\mathbf{z}_l$ is the column vector for the l -th sample in the dataset.

The number of resulting linear discriminants, m , can range from 1 to $r - 1$ (where $|\Omega_X| = r$), constituting a form of supervised dimensionality reduction of the original variable set. The maximization problem is solved by computing the first m eigenvalues of the matrix $\mathbf{S}_w^{-1} \mathbf{S}_b$, where the columns of $\mathbf{W}$ are the eigenvectors associated with these eigenvalues.

In this work, we use LDA to generate new variables that are added to the set of *dummy* variables from T_d , thereby retaining the original *dummy* variables. According to Proposition 2, simply adding the new continuous variables $\mathbf{W}^T_d(\mathbf{Y})$ to T_d would result in an equivalent transformation. To generate new, meaningful, and discrete variables, we instead follow this procedure:

- Compute the $r - 1$ continuous variables $\mathbf{W}^T_d(\mathbf{Y})$ using LDA. The computational complexity of this step is $O(Nnt + t^3)$, where $t = \min(n, N)$.
- Discretize these new variables using the Fayyad and Irani approach [32]. This step has a worst-case computational complexity of $O(N^2)$ [33] for each variable and, consequently, $O(N^2r)$ for the set of all variables.
- Convert the discretized variables into a new set of *dummy* variables using the same procedure as in the definition of T_d . The complexity of this step is $O(N)$ for each dummy variable, so the final complexity is $O(Nn')$, where n' is the number of dummy variables created in this process. This depends on the number of cases of the discretized variables, and given the possible cut points of Fayyad and Irani approach, n' is bounded by rN and the complexity total is of order $O(N^2r)$.
- Add this new set of *dummy* variables to the original set from T_d to create the final transformation, T_f . The complexity of this step depends on how a set of columns is added to an existing dataset. But, even if all the data are copied, it is bounded by the size of the added dataset which is of order $O(N^2r)$.

The final complexity of the full transformation assuming that $N \geq n$ (it is reasonable to have more cases than variables) is obtained by adding the complexities of above steps: $O(Nn^2 + n^3 + N^2r)$. Given our assumption this simplifies to $O(Nn^2 + N^2r)$. Adding logistic regression with SAGA procedure only adds terms of complexity $O(Nm')$, where m' is the total number of variables. Since m' is of order $O(n + Nr)$, we find that logistic regression is of order $O(Nn + N^2r)$. Therefore, the complexity of the full hybrid procedure is $O(Nn^2 + N^2r)$.

Example 8. One challenge associated with this procedure is the interpretability of the new variables. When using only the initial dummy variables, the logistic regression coefficients indicate the relevance of a specific parent value $Y = y$ to the probability of X_i , reflecting both the direction and strength of its influence. Defining linear combinations and subsequently discretizing them could be perceived as obscuring

Ictus probability according to Score CHA₂DS₂-VASc (Olesen et al.)

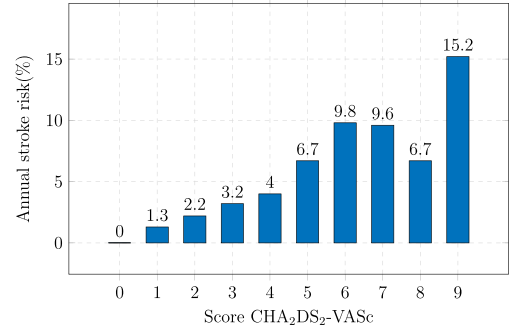


Fig. 1. Probability as a function of the risk score.

Table 1

Bayesian networks for the experiments.

Name	Nodes	Arcs	Parameters	Zero Proportion	N. Conditional
alarm	37	46	509	0.0	2
barley	48	84	114,005	0.0	28
diabetes	413	602	429,409	0.826	241
hailfinder	56	66	2656	0.074	6
hepar2	70	123	1453	0.0	6
insurance	27	52	1008	0.347	4
link	724	1125	14,211	1.0	118
mildew	35	46	540,150	0.942	17
pathfinder	109	195	72,079	0.604	54
water	32	66	10,083	0.696	8
win95pts	76	112	574	0.596	2
munin	1041	1397	80,592	0.717	227

these individual effects. Nevertheless, this approach is common across many fields; for instance, risk scores in medicine often follow this logic. Consider the CHA₂DS₂-VAsC score for stroke risk in patients with Atrial Fibrillation (AFib) [34]. This score is calculated by summing points for various factors: Congestive heart failure (1), Hypertension (1), Age ≥ 75 (2), Diabetes (1), Stroke history (2), Vascular disease (1), Age 65–74 (1), and Female sex (1). This represents a linear combination of dummy variables. The resulting score is then mapped to stroke probabilities [35], as shown in Fig. 1. We may note that, once discretized into integer values, the relationship between the score and probability is neither linear nor exponential. Moreover, it is not necessarily monotonic: scores of 7 and 8 actually show a lower stroke probability than a score of 6. This example demonstrates that combining linear combinations with discretization can break both linearity and monotonicity, a process that our procedure aims to generalize.

It is also possible to combine variable generation methods. For instance, one can include the variables from both $T_{d \oplus r}$ and T_f , a combination represented by $T_{d \oplus r \oplus f}$. Other combinations are also conceivable. For example, the variables in $T_{d \oplus r}$ could be incorporated into the definition of the new variables computed by logistic regression, or the variables defined in T_f could be used to construct new variables through the XOR combination. However, these variations have not been tested in this paper.

5. Experiments

To evaluate the proposed methods, we conducted an extensive experimental study. A set of Bayesian networks was selected from the bnlearn repository [36,37] (all networks in the repository with discrete variables excluding those with a node count below 20). The used networks and their main properties are given in Table 1. In each network, we considered all nodes with more than one parent and a CPT larger than 64 entries. In some networks, no nodes satisfied these criteria; therefore, they were excluded from the initial selection. The final set of networks is shown in Table 1. This table reports, for each network, the number of

Table 2Average log likelihood as a function of α regularization..

Network	$\alpha = 0.01$	$\alpha = 0.05$	$\alpha = 0.1$	$\alpha = 0.5$	$\alpha = 1$	$\alpha = 2$	$\alpha = 4$
<i>alarm</i>	-0.6861	-0.6728	-0.6669	-0.6601	-0.6653	-0.6789	-0.7107
<i>barley</i>	-1.2185	-1.1895	-1.1783	-1.2166	-1.2721	-1.3364	-1.4398
<i>diabetes</i>	-0.9997	-0.8931	-0.8354	-0.8679	-0.9419	-1.0450	-1.2066
<i>hailfinder</i>	-0.8523	-0.8415	-0.8344	-0.8247	-0.8285	-0.8504	-0.9004
<i>hepar2</i>	-0.9634	-0.9547	-0.9482	-0.9329	-0.9304	-0.9287	-0.9317
<i>insurance</i>	-0.4267	-0.4079	-0.3986	-0.3888	-0.3949	-0.4101	-0.4419
<i>link</i>	-0.5460	-0.5465	-0.5472	-0.5539	-0.5645	-0.5872	-0.6358
<i>mildew</i>	-0.8988	-0.8503	-0.8285	-0.8277	-0.8580	-0.9137	-1.0028
<i>munin</i>	-0.1558	-0.1384	-0.1293	-0.1235	-0.1415	-0.1873	-0.2452
<i>pathfinder</i>	-0.3444	-0.2882	-0.2642	-0.2227	-0.2334	-0.2726	-0.3274
<i>water</i>	-0.2690	-0.2563	-0.2497	-0.2394	-0.2520	-0.2741	-0.3015
<i>win95pts</i>	-0.3182	-0.3097	-0.3070	-0.2911	-0.2923	-0.3114	-0.3541

Table 3Average size as a function of α regularization.

Network	$\alpha = 0.01$	$\alpha = 0.05$	$\alpha = 0.1$	$\alpha = 0.5$	$\alpha = 1$	$\alpha = 2$	$\alpha = 4$
<i>alarm</i>	24.07	22.70	21.53	17.17	14.17	10.90	8.17
<i>barley</i>	125.92	112.27	105.81	86.43	71.68	52.38	30.32
<i>diabetes</i>	220.57	161.61	143.50	109.30	77.41	51.35	29.62
<i>diabetes</i>	27.07	24.13	23.83	23.07	22.37	19.97	15.70
<i>hepar2</i>	24.10	22.10	21.57	17.70	13.17	8.30	4.97
<i>insurance</i>	30.79	26.08	24.75	19.42	15.58	11.79	8.79
<i>link</i>	22.66	20.95	20.81	20.14	18.64	16.10	14.31
<i>mildew</i>	90.56	79.89	74.50	61.50	47.61	33.67	20.17
<i>munin</i>	29.99	23.14	21.07	16.70	12.26	8.55	6.12
<i>pathfinder</i>	89.87	54.82	45.62	29.89	17.84	10.73	7.02
<i>water</i>	39.48	33.04	30.08	22.92	17.60	11.12	6.04
<i>win95pts</i>	8.00	8.00	8.00	7.93	7.59	7.34	6.66

nodes, arcs, and parameters, as well as the proportion of zero values in the CPTs (restricted to those with a size greater than 64) and the number of CPTs with more than 64 entries (N. Conditional). For networks with few CPTs of this size, we repeated the estimation of the conditional probabilities until each network reached a count of at least 30. The proportion of zeros is significant as it is notably high in certain networks; for instance, in the *link* network, the proportion is 1.0. In such cases, all CPTs are deterministic, consisting exclusively of 0s and 1s.

We implemented the experiments in Python.¹ The `pgmpy` package [38] was used for handling Bayesian networks and the standard estimation procedure, while `scikit-learn` [39] was employed for logistic regression. The probability tree estimation procedure was implemented directly by the authors.

We conducted a preliminary experiment to assess the influence of the regularization parameter α and the Bayesian estimation parameter S . For each network, we simulated a training sample of 500 instances and a test sample of 1000 instances. The analysis was restricted to dummy variables only. For regularization, we employed logistic regression across various values of α ; for Bayesian estimation, we estimated both CPTs and PTs using different values of S .

Tables 2 and 3 depict, respectively, the average log likelihoods on the test data and the model sizes (number of non-zero parameters) for logistic regression. We observe that the size of the model decreases as regularization strengthens. Concerning log-likelihood performance, the optimal α value depends on the network; however, in general, both very low ($\alpha = 0.01$) and very large ($\alpha = 4$) yield poor results. Only in the *link* network $\alpha = 0.01$ does lead to adequate performance, though this is a unique case where all estimated conditional probabilities are extreme (0 or 1), and the differences compared to other regularization levels remain small. Table 4 presents the results of Friedman non-parametric tests, including significant pairs identified by pairwise Wilcoxon signed-rank tests with Holm-Bonferroni correction. Most of these differences

Table 4

Friedman test and significant post-hoc comparisons per network.

Network	Friedman p	Significant Pairs α values (Adjusted $p < 0.01$)
<i>alarm</i>	4.0e-21	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-1, 0.01-4, 0.05-0.1, 0.05-0.5, 0.05-4, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>barley</i>	7.7e-15	0.01-0.05, 0.01-4, 0.05-1, 0.05-2, 0.05-4, 0.1-1, 0.1-2, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>diabetes</i>	4.8e-138	0.01-0.05, 0.01-0.1, 0.01-1, 0.01-2, 0.01-4, 0.05-0.1, 0.05-0.5, 0.05-1, 0.05-2, 0.05-4, 0.1-0.5, 0.1-1, 0.1-2, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>hailfinder</i>	1.8e-18	0.01-4, 0.05-4, 0.1-4, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>hepar22</i>	1.3e-22	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-1, 0.01-2, 0.01-4, 0.05-0.1, 0.05-0.5, 0.05-1, 0.05-2, 0.05-4, 0.1-0.5, 0.1-1, 0.1-2, 0.5-1
<i>insurance</i>	6.2e-17	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-1, 0.05-0.1, 0.05-0.5, 0.05-4, 0.1-0.5, 0.1-4, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>link</i>	1.1e-149	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-1, 0.01-2, 0.01-4, 0.05-0.1, 0.05-0.5, 0.05-1, 0.05-2, 0.05-4, 0.1-0.5, 0.1-1, 0.1-2, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>mildew</i>	2.5e-13	0.01-0.05, 0.01-0.1, 0.05-0.1, 0.05-4, 0.1-2, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>munin</i>	2.3e-109	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-2, 0.01-4, 0.05-0.1, 0.05-1, 0.05-2, 0.05-4, 0.1-1, 0.1-2, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>pathfinder</i>	3.8e-24	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-1, 0.05-0.1, 0.05-0.5, 0.05-1, 0.1-0.5, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>water</i>	7.9e-23	0.01-0.05, 0.01-0.1, 0.01-0.5, 0.01-1, 0.01-4, 0.05-0.1, 0.05-0.5, 0.05-2, 0.05-4, 0.1-2, 0.1-4, 0.5-1, 0.5-2, 0.5-4, 1-2, 1-4, 2-4
<i>win95pts</i>	2.5e-07	0.01-0.05, 0.01-0.1, 0.05-0.1, 0.05-4, 0.1-4, 0.5-4, 1-2, 1-4, 2-4

are statistically significant. Based on these findings, we selected $\alpha = 0.5$ for our experiments.

Regarding the S parameter, Tables 5 and 6 show the average log-likelihoods for the different values of S . As expected, given the dimensions of the estimated tables, the best performance is achieved with the highest S values. Consequently, we adopted these values for both the tables and the trees.

The main experiment compares the different methods with different sample sizes. For each network, random samples were generated with sizes of 500, 5000, and 10,000. Then, for each node satisfying the inclusion conditions (more than one parent and CPT size greater than 64), different procedures were applied to estimate the CPTs:

- **BT** (Bayesian Table estimation): A Bayesian approach to estimate the full CPT according to Expression (1), with a global sample size of $S = 15$.
- **TR** (Probability TRee estimation): A probability tree approach to estimate the conditional probabilities, with a global sample size of $S = 15$.

¹ Data and source code to replicate our experiments can be found at <https://github.com/serafinmoral/anomalies>

Table 5
Average Log-Table values per network.

Network	S = 1	S = 2	S = 5	S = 10	S = 15
<i>alarm</i>	−0.6473	−0.6415	−0.6348	−0.6310	−0.6296
<i>barley</i>	−1.7318	−1.6460	−1.5343	−1.4525	−1.4066
<i>diabetes</i>	−0.5346	−0.5163	−0.4942	−0.4807	−0.4753
<i>hailfinder</i>	−1.3105	−1.2459	−1.1620	−1.1007	−1.0665
<i>hepar22</i>	−1.0187	−1.0033	−0.9835	−0.9694	−0.9618
<i>insurance</i>	−0.5288	−0.5142	−0.4967	−0.4864	−0.4825
<i>link</i>	−0.0016	−0.0032	−0.0080	−0.0158	−0.0236
<i>mildew</i>	−1.2548	−1.1954	−1.1191	−1.0648	−1.0356
<i>munin</i>	−0.1124	−0.1088	−0.1052	−0.1038	−0.1040
<i>pathfinder</i>	−0.4354	−0.4166	−0.3926	−0.3757	−0.3667
<i>water</i>	−0.3641	−0.3522	−0.3366	−0.3251	−0.3185
<i>win95pts</i>	−0.1207	−0.1145	−0.1068	−0.1015	−0.0989

Table 6
Average Log-Tree values per network.

Network	S = 1	S = 2	S = 5	S = 10	S = 15
<i>alarm</i>	−0.6611	−0.6531	−0.6467	−0.6425	−0.6392
<i>barley</i>	−1.2404	−1.2311	−1.1986	−1.1862	−1.1758
<i>diabetes</i>	−1.2331	−1.1987	−1.1303	−1.0489	−1.0235
<i>hailfinder</i>	−0.9229	−0.9154	−0.9028	−0.8930	−0.8903
<i>hepar2</i>	−0.9341	−0.9297	−0.9272	−0.9277	−0.9272
<i>insurance</i>	−0.5082	−0.5003	−0.4902	−0.4860	−0.4800
<i>link</i>	−0.0737	−0.0284	−0.0115	−0.0157	−0.0234
<i>mildew</i>	−1.0535	−1.0216	−0.9991	−0.9715	−0.9648
<i>munin</i>	−0.1410	−0.1370	−0.1314	−0.1285	−0.1286
<i>pathfinder</i>	−0.3567	−0.3560	−0.3554	−0.3589	−0.3575
<i>water</i>	−0.2541	−0.2541	−0.2525	−0.2525	−0.2542
<i>win95pts</i>	−0.1338	−0.1257	−0.1159	−0.1106	−0.1085

- **LRT_a** (Logistic Regression using T_a transformation): A logistic regression approach using variables from a transformation T_a . We test four transformations: T_d (denoted **LRT_d**), $T_{d \oplus r}$ with $M = 2$ (denoted **LRT_{d \oplus}**), T_l (denoted **LRT_l**), and the combination $T_{d \oplus r \oplus l}$ (denoted **LRT_{d \oplus r \oplus l}**). In the discretization process we have limited the number of cases for each discretized variable to 20.
- **TRT_a** (Probability TRee estimation using T_a transformation): A probability tree estimation using variables from a transformation T_a , with the same set of transformations as for logistic regression.

We discarded instances of the conditional probabilities of X_i given its parents where not all possible outcomes of X_i were represented in the sample (i.e., a variable value that never appears in the dataset). This specific situation caused errors in the logistic regression libraries utilized. Table 7 summarizes the percentages of discarded conditional probabilities; notably, in approximately half of the networks, no cases were discarded. In some networks, such as *water*, certain variables have states with such low probabilities that they fail to appear during sampling. This phenomenon is related to the marginal probability of X_i rather than the number of parent variables—the latter being the primary challenge in estimating conditional probabilities. Therefore, we do not expect this exclusion to have a significant impact on the final results.

To evaluate the learned models, an additional random test sample of 10,000 instances was generated for each network. The primary evaluation metric is the average log-likelihood on the test set. We also report the average number of non-zero parameters for each model as a measure of model size (including the percentage relative to the full CPT model to assess the degree of compression). The results are presented in Tables 8 and 9. We performed Friedman non-parametric tests with post-hoc Wilcoxon signed-rank tests for every network and sample size, applying Holm-Bonferroni corrections. Table 10 shows the Friedman test p-values and the resulting ranking of methods; notably, methods within the same group are statistically tied ($\alpha = 0.01$). Additionally,

Table 7
Proportions of discarded conditional probabilities per network and sample size.

Network	500	5000	10,000
<i>alarm</i>	0.000	0.000	0.000
<i>barley</i>	0.371	0.203	0.142
<i>diabetes</i>	0.209	0.197	0.195
<i>hailfinder</i>	0.000	0.000	0.000
<i>hepar2</i>	0.000	0.000	0.000
<i>insurance</i>	0.147	0.025	0.000
<i>link</i>	0.000	0.000	0.000
<i>mildew</i>	0.470	0.470	0.441
<i>pathfinder</i>	0.165	0.036	0.030
<i>water</i>	0.797	0.750	0.750
<i>win95pts</i>	0.000	0.000	0.000
<i>munin</i>	0.484	0.200	0.189

Table 11 provides the p-values for the comparison between **LRT_l** and other procedures.

- The traditional Bayesian table estimation (**BT**) performs poorly when sample sizes are small relative to the CPT size. Some specific networks and their results are discussed below:
 - In the *barley* network, with an average CPT size larger than 3000,² The log-likelihood is very low, especially with small samples (−1.4259 for a sample of 500). The log-likelihood consistently increases with the sample size.
 - Similarly, **BT** performs poorly on the *mildew* network, which also features very large CPTs.
 - Despite having large CPTs (over 1500 entries), **BT** achieves better results on the *diabetes* network. This is likely because the high proportion of zeros (0.826) in the CPTs reduces the effective number of parameters to be learned.

BT also performs poorly on *hailfinder* when the sample size is small, where the CPTs are not as large (252 entries) but the proportion of zeros is negligible.

- The performance of probability trees (**TR**) relative to probability tables (**BT**) depends on both the network and the sample size, though **TR** consistently utilizes far fewer parameters. **BT** leads to superior results in *alarm*, *diabetes*, and *munin*, whereas both methods perform similarly in the *pathfinder* and *win95pts* networks. In all other networks, **TR** outperforms **BT**. We may note that, in every case, probability trees require fewer parameters than probability tables. For example, in *water* with a sample size of 500, the log-likelihood of **BT** is −0.3227, while **TR** achieves −0.2583 using only 0.7% of the parameters of **BT**. As the sample size increases, the gap in log-likelihood diminishes, while the number of parameters in the probability tree grows.
- Logistic regression with only *dummy* variables (**LRT_d**) generally performs poorly, as it is only well-suited to specific cases. This model is equivalent to Naive Bayes [40] and thus performs well only when the parent variables are conditionally independent given the child X_i . It outperforms the basic **BT** in *water*, *hepar2*, *barley*, *mildew*, and *pathfinder*. In other networks, such as *hailfinder*, *insurance*, and *water*, it is better than **BT** for small samples, but the inherent bias of the procedure makes **BT** superior for large samples. In the remaining cases, its performance is worse than **BT**. One advantage is that its number of parameters is always lower than that of **BT** or **TR**.

² Theoretically, the average table size should be invariant to sample size. However, variations arise because nodes are discarded from a given sample if any state of the child variable is unobserved. The *barley* network has variables with high cardinality (up to 60 states), making this situation more frequent.

Table 8
Average log likelihood values.

Net	Size	BT	TR	LRT_d	TRT_d	$LRT_{d\oplus}$	$TRT_{d\oplus}$	LRT_l	TRT_l	$LRT_{d\oplus l}$	$TRT_{d\oplus l}$
<i>alarm</i>	500	-0.6178	-0.6332	-0.6897	-0.6378	-0.6490	-0.6297	-0.6198	-0.6282	-0.6188	-0.6282
	5000	-0.5987	-0.6006	-0.6742	-0.6005	-0.6247	-0.5991	-0.5984	-0.5994	-0.5984	-0.5991
	10,000	-0.5970	-0.5981	-0.6731	-0.5981	-0.6237	-0.5974	-0.5968	-0.5972	-0.5968	-0.5972
<i>barley</i>	500	-1.4259	-1.2057	-1.1481	-1.2536	-1.0799	-1.2078	-0.9085	-0.9741	-0.9075	-0.9751
	5000	-1.0080	-0.8445	-0.8348	-0.9014	-0.8248	-0.8898	-0.7747	-0.7881	-0.7745	-0.7889
	10,000	-0.8491	-0.7652	-0.7658	-0.8005	-0.7617	-0.7926	-0.7265	-0.7379	-0.7264	-0.7379
<i>diabetes</i>	500	-0.4630	-1.0127	-0.9652	-1.2558	-0.8123	-1.1365	-0.3948	-0.6770	-0.3947	-0.6667
	5000	-0.3062	-0.4212	-0.4953	-0.5737	-0.4804	-0.5126	-0.2872	-0.3614	-0.2873	-0.3612
	10,000	-0.2875	-0.3332	-0.4155	-0.4338	-0.4237	-0.3975	-0.2763	-0.3212	-0.2763	-0.3199
<i>hailfinder</i>	500	-1.0455	-0.8793	-0.8348	-0.8942	-0.8352	-0.8933	-0.8412	-0.8663	-0.8417	-0.8663
	5000	-0.8272	-0.8074	-0.8122	-0.8113	-0.8122	-0.8112	-0.7954	-0.8025	-0.7954	-0.8024
	10,000	-0.8004	-0.7996	-0.8108	-0.7994	-0.8108	-0.7993	-0.7906	-0.7994	-0.7906	-0.7948
<i>hepar2</i>	500	-0.9675	-0.9334	-0.9310	-0.9337	-0.9308	-0.9344	-0.9320	-0.9354	-0.9321	-0.9356
	5000	-0.9205	-0.9142	-0.9169	-0.9136	-0.9169	-0.9136	-0.9130	-0.9137	-0.9130	-0.9137
	10,000	-0.9152	-0.9115	-0.9158	-0.9113	-0.9158	-0.9113	-0.9105	-0.9109	-0.9105	-0.9109
<i>insurance</i>	500	-0.4885	-0.4793	-0.4556	-0.4849	-0.4571	-0.4855	-0.4601	-0.4807	-0.4614	-0.4809
	5000	-0.3526	-0.3520	-0.3554	-0.3534	-0.3554	-0.3534	-0.3508	-0.3550	-0.3508	-0.3550
	10,000	-0.3494	-0.3499	-0.3540	-0.3502	-0.3540	-0.3502	-0.3483	-0.3506	-0.3483	-0.3506
<i>link.bif</i>	500	-0.0235	-0.0234	-0.5450	-0.0430	-0.3132	-0.0385	-0.0174	-0.0217	-0.0164	-0.0214
	5000	-0.0023	-0.0023	-0.5197	-0.0022	-0.5175	-0.0022	-0.0016	-0.0022	-0.0016	-0.0022
	10,000	-0.0011	-0.0011	-0.5184	-0.0011	-0.5184	-0.0011	-0.0008	-0.0011	-0.0008	-0.0011
<i>mildew</i>	500	-1.0406	-0.9710	-0.8524	-0.9784	-0.8394	-0.9328	-0.8540	-0.8802	-0.8475	-0.8694
	5000	-0.7707	-0.7868	-0.7523	-0.7977	-0.7502	-0.7875	-0.7530	-0.7806	-0.7523	-0.7764
	10,000	-0.7478	-0.7667	-0.7458	-0.7766	-0.7449	-0.7685	-0.7421	-0.7651	-0.7417	-0.7608
<i>munin</i>	500	-0.1232	-0.1406	-0.1444	-0.1610	-0.1378	-0.1578	-0.1304	-0.1477	-0.1296	-0.1475
	5000	-0.0922	-0.0941	-0.0962	-0.1000	-0.0942	-0.0992	-0.0921	-0.0969	-0.0920	-0.0970
	10,000	-0.0910	-0.0922	-0.0940	-0.0947	-0.0924	-0.0949	-0.0909	-0.0937	-0.0908	-0.0937
<i>pathfinder</i>	500	-0.3277	-0.3094	-0.2315	-0.2562	-0.2320	-0.2481	-0.2541	-0.2601	-0.2493	-0.2576
	5000	-0.2352	-0.2347	-0.2221	-0.2334	-0.2221	-0.2303	-0.2223	-0.2241	-0.2221	-0.2238
	10,000	-0.2147	-0.2155	-0.2078	-0.2164	-0.2076	-0.2150	-0.2070	-0.2085	-0.2070	-0.2083
<i>water</i>	500	-0.3227	-0.2583	-0.2457	-0.2630	-0.2446	-0.2614	-0.2485	-0.2572	-0.2481	-0.2572
	5000	-0.2297	-0.2228	-0.2175	-0.2210	-0.2173	-0.2218	-0.2166	-0.2198	-0.2164	-0.2199
	10,000	-0.2154	-0.2108	-0.2103	-0.2118	-0.2101	-0.2121	-0.2088	-0.2110	-0.2087	-0.2110
<i>win95pts</i>	500	-0.0940	-0.1012	-0.2888	-0.1012	-0.1963	-0.1038	-0.1225	-0.0910	-0.1156	-0.0923
	5000	-0.0665	-0.0657	-0.2871	-0.0657	-0.1543	-0.0660	-0.0696	-0.0652	-0.0686	-0.0652
	10,000	-0.0645	-0.0639	-0.2868	-0.0639	-0.1525	-0.0640	-0.0674	-0.0633	-0.0663	-0.0630

- Between the two proposed variable generation methods, *LDA* (LRT_l) provides the most significant improvements. The *XOR*-based variables ($LRT_{d\oplus}$) improve upon or match the performance of LRT_d , but the gains for LRT_l are substantially larger. In some networks, such as *barley*, *diabetes*, *link*, or *win95pts*, the difference is very important, whereas in others both procedures produce similar results, or $LRT_{d\oplus}$ is slightly better. The model sizes depend on the network, but the increase in parameters with LRT_l is typically more pronounced as the sample size grows because the discretization of the linear discriminant functions produces more intervals. $LRT_{d\oplus}$ performs significantly better than LRT_l only in the *mildew* network with 500 and 1000 samples and in *pathfinder* network with 500 samples. In almost all the other networks and sample sizes, LRT_l demonstrates superior performance.
- Combining both methods ($LRT_{d\oplus l}$) never degrades performance compared to LRT_l alone and occasionally yields slight improvements. We observe significant differences in *link* (sample size 500), *mildew*, and *water* (sample sizes 5000, 10000). The sizes of both models are quite similar.
- The definition of *dummy* or artificial variables in trees always decreases the sizes of the estimated trees. In some cases, this decrease is significant. For example, in the *diabetes* network, with a sample size of 500, an average size of 249.0 is obtained for basic trees, and a size of 85.2 for TRT_l . Even with this smaller size, it improves the log-likelihood performance (-0.6770 vs -1.0127). For this transformation, T_l logistic regression always performs better or similarly to probability trees, except for *insurance* and *munin*. These are situations in which generic Bayesian estimation produces

good results compared to logistic regression with the addition of artificial variables. Nonetheless, these are particular cases in which the estimation of probability tables is simple. In fact, in both cases, the average table size is small and the number of 0 values is very high in *munin*. This makes the estimation by probability tables or probability trees very simple, as cases with probability 0 never appear in the learning sample nor in the test sample.

- Both logistic regression and probability tree-based methods require significantly fewer parameters than probability tables. This efficiency is particularly pronounced in probability trees when incorporating additional variables derived from discriminant analysis.

6. Conclusions and future work

The central contribution of this research is the development of models that capture the dependence of a categorical variable on a set of parents while requiring significantly fewer parameters than a full Conditional Probability Table. Current state-of-the-art software tools for Bayesian networks, such as pgmpy [41] or bnlearn [37], primarily rely on full table estimation for discrete conditional distributions. While this approach is feasible when the number of parents is small, it becomes computationally and statistically intractable as the parent set grows. For continuous variables, established models like the Conditional Gaussian framework offer a much more parsimonious parameterization. We contend that developing similarly efficient models for discrete variables is a critical yet underdeveloped area. The methodology proposed in this paper represents a step toward addressing this gap.

Table 11 (continued)

Network	Size	BT	TR	LRT _d	TRT _d	LRT _{d⊕}	TRT _{d⊕}	TRT _l	LRT _{d⊕l}	TRT _{d⊕l}
pathfinder	10,000	1.6e-10	5.7e-09	0.114	5.8e-07	0.254	4.4e-06	8.5e-04	0.633	0.005
water	500	7.6e-06	0.018	0.167	3.3e-04	0.043	0.002	8.4e-04	0.609	8.4e-04
water	5000	1.2e-07	1.2e-07	0.016	1.2e-07	0.101	1.2e-07	1.2e-07	1.6e-05	1.2e-07
water	10,000	6.0e-08	1.8e-05	2.0e-06	6.0e-07	1.5e-05	3.0e-07	3.3e-06	1.5e-05	2.0e-06
win95pts	500	0.006	0.027	3.7e-09	0.027	7.7e-07	0.039	7.7e-07	0.046	1.0e-05
win95pts	5000	8.9e-05	1.4e-06	1.9e-09	1.4e-06	1.9e-09	6.0e-06	6.1e-08	7.3e-04	1.3e-07
win95pts	10,000	4.7e-08	1.0e-07	1.9e-09	1.0e-07	1.9e-09	4.7e-08	5.6e-09	3.5e-08	1.9e-09

A possible application of this procedure is the automatic generation of medical scores, generalizing the case of [Example 8](#).

CRedit authorship contribution statement

Serafin Moral: Supervision, Software, Conceptualization. **Serafin Moral-García:** Writing – original draft, Validation, Formal analysis. **Andrés Cano:** Project administration, Methodology, Investigation. **Manuel Gómez-Olmedo:** Writing – review & editing, Visualization, Methodology.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Gemini to correct typographical and grammatical errors, improve the quality of the writing, and generate the tables. After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was jointly supported by the Spanish Ministry of Science, Innovation, and Universities under projects PID2022-139293NB-C32 TOPAI and PID2024-159012NA-I00, and the European Regional Development Fund (FEDER). We appreciate the reviewers' insightful comments, which have helped us significantly improve the quality of our work.

Appendix A. Proofs

Proof of Proposition 1. The matrix A_s has an extra column for Z_c . Its first columns are identical to A , and the last column corresponds to C . Therefore,

$$A_s T_s(y) = (A \ C) (T(y) \ 1) = AT(y) + C. \quad \square$$

Proof of Proposition 2. Let the new variable be Z_n , defined as $Z_n = \sum_{Z \in T_2} \alpha_Z Z$. Adding Z_n to T_1 yields T_{1n} . The parameter matrix for the new transformation can be written as $A_{1n} = (A_1 \ a_{Z_n})$, where a_{Z_n} is the column corresponding to Z_n . The resulting matrix B_{1n} is computed as follows:

$$A_{1n} T_{1n}(y) = (A_1 \ a_{Z_n}) \begin{pmatrix} T_1(y) \\ Z_n(y) \end{pmatrix} = A_1 T_1(y) + a_{Z_n} Z_n(y). \quad (A.1)$$

Substituting the definition of Z_n as a linear combination of the variables in T_2 , we obtain:

$$A_{1n} T_{1n}(y) = A_1 T_1(y) + a_{Z_n} \sum_{Z \in T_2} \alpha_Z Z(y) \quad (A.2)$$

Next, define a matrix A_2^* with the same dimensions as A_2 , in which each column corresponding to a variable $Z \in T_2$ is given by $\alpha_Z a_{Z_n}$. Let

$A_{2n} = A_2 + A_2^*$ (an element-wise sum). The resulting transformation can be expressed as:

$$\begin{aligned} A_{2n} T_2(y) &= A_2 T_2(y) + A_2^* T_2(y) = A_2 T_2(y) + \sum_{Z \in T_2} \alpha_Z a_{Z_n} Z(y) \\ &= A_2 T_2(y) + a_{Z_n} \sum_{Z \in T_2} \alpha_Z Z(y). \end{aligned} \quad (A.3)$$

Since $T_1 \leq T_2$, we can replace the term $A_1 T_1(y)$ in [Eq. \(A.2\)](#) with an equivalent $A_2 T_2(y)$. A comparison of [Eqs. \(A.2\)](#) and [\(A.3\)](#) shows that $T_{1n} \leq T_2$. $\square$

Proof of Proposition 3. Under T_c , there is one variable for each configuration, so $Z_c = \{Z_1, \dots, Z_L\}$, with $\Omega_{Z_m} = \{0, 1\}$ for all Z_m ($m = 1, \dots, L$). As shown in [Example 5](#), $B_c = A_c$, since $T_c(Y) = I_L$. For a specific configuration y_m , the corresponding vector $B_c(m)$ is:

$$B_c^T(m) = \begin{pmatrix} \gamma_{1m} \\ \dots \\ \gamma_{(r-1)m} \end{pmatrix} \quad (A.4)$$

Under T_{ds} , the set of variables H consists of subsets of the underlying dummy variables. For a given configuration y_m , we first identify the set of active dummy variables, Z_{dR} : variables $Z_{jk} \in Z$ such that $Z_{jk} = 1$ according to y_m . Next, we identify the collection of subsets in H that are composed entirely of variables included in Z_{dR} ; let this collection be H_{dR} . In the transformed vector $T_{ds}(y_m)$, the entries corresponding to these active subsets in H_{dR} are 1 (and 0 otherwise). The column vector $B_{ds}(m)$ can then be expressed as the sum of the parameter vectors for these active subsets:

$$B_{ds}^T(m) = \begin{pmatrix} \sum_{H_{sj} \in H_{dR}} \delta_{1s_j} \\ \dots \\ \sum_{H_{sj} \in H_{dR}} \delta_{(r-1)s_j} \end{pmatrix} \quad (A.5)$$

For the two transformations to be equivalent, the resulting vectors from [Eqs. \(A.4\)](#) and [\(A.5\)](#) must be equal for all configurations m :

$$\begin{pmatrix} \gamma_{1m} \\ \dots \\ \gamma_{(r-1)m} \end{pmatrix} = \begin{pmatrix} \sum_{H_{sj} \in H_{dR}} \delta_{1s_j} \\ \dots \\ \sum_{H_{sj} \in H_{dR}} \delta_{(r-1)s_j} \end{pmatrix}$$

These equalities allow us to compute the values of matrix A_c from the parameters in A_{ds} , but also allow us to compute an equivalent matrix A_{ds} from a matrix A_c . If H_{s_k} is the subset corresponding to Z_{dR} , then we can carry out a recursive computation of δ values in increasing size of H_{s_j} subsets by considering:

$$\delta_{is_k} = \gamma_{im} - \sum_{H_{sj} \in H_{dR} \setminus \{Z_{dr}\}} \delta_{is_j}. \quad \square$$

Proof of Proposition 4. Note that [Eq. \(18\)](#) expands the XOR of variables into a linear combination of products over all non-empty subsets of those variables. We generalize this expression for any set H . Let

$\mathbf{H} = \{Z_1, \dots, Z_n\}$, let $\mathbf{S}_k$ be the collection of all subsets of $\mathbf{H}$ with size k , and let $\mathbf{S}_{\mathbf{H}}$ be the set of all non-empty subsets of $\mathbf{H}$:

$$\mathbf{S}_{\mathbf{H}} = \mathbf{S}_1 \cup \mathbf{S}_2 \cup \dots \cup \mathbf{S}_{n-1} \cup (\mathbf{S}_n = \mathbf{H}) = 2^{\mathbf{H}} \setminus \emptyset. \quad (\text{A.6})$$

Using this notation, the XOR over all variables in $\mathbf{H}$ can be written as:

$$\bigoplus_{Z_j \in \mathbf{H}} Z_j = \sum_{\mathbf{S}_i \in \mathbf{S}_{\mathbf{H}}} (-2)^{(|\mathbf{S}_i|-1)} \prod_{Z_i \in \mathbf{S}_i} Z_i. \quad (\text{A.7})$$

We use induction to prove that Eq. (A.7) holds for any number of variables. The base case, $|\mathbf{H}| = 1$, is trivial. Assume that it is valid for $|\mathbf{H}| = n$. We now check the inductive step by adding a new variable Z_+ to $\mathbf{H}$, creating a new set $\mathbf{H}_+$ of size $n+1$. Applying the definition from Eq. (17) we have:

$$\left(\bigoplus_{Z_j \in \mathbf{H}} Z_j \right) \oplus Z_+ = \left(\bigoplus_{Z_j \in \mathbf{H}} Z_j \right) + Z_+ - 2 \left(\bigoplus_{Z_j \in \mathbf{H}} Z_j \right) Z_+. \quad (\text{A.8})$$

Assuming the induction hypothesis and applying the XOR substitution for the variables in $\mathbf{H}$, we have:

$$\left[\sum_{\mathbf{S}_i \in \mathbf{S}_{\mathbf{H}}} (-2)^{(|\mathbf{S}_i|-1)} \prod_{Z_i \in \mathbf{S}_i} Z_i + Z_+ \right] + \left[-2 \left(\sum_{\mathbf{S}_i \in \mathbf{S}_{\mathbf{H}}} (-2)^{(|\mathbf{S}_i|-1)} \prod_{Z_i \in \mathbf{S}_i} Z_i \right) Z_+ \right]. \quad (\text{A.9})$$

The first two terms of Eq. (A.9) contain all subsets of $\mathbf{H}_+$ with a single variable ($\mathbf{S}_1 \subset \mathbf{S}_{\mathbf{H}_+}$) and all those of greater length that do not include Z_+ , that is: $\{\mathbf{S}_2, \dots, \mathbf{S}_n\} \subset \mathbf{S}_{\mathbf{H}}$. Then, the first two terms within the first pair of brackets can be written as:

$$\sum_{\substack{\mathbf{S}_i \in (\{\mathbf{S}_1 \subset \mathbf{S}_{\mathbf{H}_+}\} \cup \\ \{(\mathbf{S}_2, \dots, \mathbf{S}_n) \subset \mathbf{S}_{\mathbf{H}}\})}} (-2)^{(|\mathbf{S}_i|-1)} \prod_{Z_i \in \mathbf{S}_i} Z_i. \quad (\text{A.10})$$

For its part, the third term (within the second part of brackets) represents the inclusion of Z_+ in all subsets of $\mathbf{S}_{\mathbf{H}}$. Then, the term represents elements of $\mathbf{S}_{\mathbf{H}_+}$ with two or more variables and including Z_+ . By adding a new variable to the subsets, the coefficient (-2) can be incorporated into the general expression, taking into account that subsets have increased in size. In short, it could be expressed as follows:

$$\sum_{\substack{\mathbf{S}_i \in (\{\mathbf{S}_2, \dots, \mathbf{S}_{n+1}\} \subset \mathbf{S}_{\mathbf{H}_+}) \\ Z_+ \in \mathbf{S}_i}} (-2)^{(|\mathbf{S}_i|-1)} \prod_{Z_i \in \mathbf{S}_i} Z_i. \quad (\text{A.11})$$

Considering Eqs. (A.10) and (A.11), it can be seen that their sum comprises all possible subsets in $\mathbf{S}_{\mathbf{H}_+}$:

$$\begin{aligned} & [\mathbf{S}_1 \subset \mathbf{S}_{\mathbf{H}_+}] \cup [(\mathbf{S}_i \in (\{\mathbf{S}_2, \dots, \mathbf{S}_n\}) \subset \mathbf{S}_{\mathbf{H}})] \cup \\ & [((\mathbf{S}_j | Z_+ \in \mathbf{S}_j) \in (\{\mathbf{S}_2, \dots, \mathbf{S}_n\})) \subset \mathbf{S}_{\mathbf{H}_+}] = \mathbf{S}_{\mathbf{H}_+}. \end{aligned} \quad (\text{A.12})$$

Therefore, the complete expression in Eq. (A.9) can be rewritten as:

$$\sum_{\mathbf{S}_i \in \mathbf{S}_{\mathbf{H}_+}} (-2)^{(|\mathbf{S}_i|-1)} \prod_{Z_i \in \mathbf{S}_i} Z_i. \quad (\text{A.13})$$

This equation proves that $\bigoplus_{Z_j \in \mathbf{H}} Z_j$ is a linear combination of variables $\prod_{Z_j \in \mathbf{H}_s} Z_j$ where $\mathbf{H}_s \subseteq \mathbf{H}$. As T_{1n} is the result of adding variables to T_1 we have that $T_2 \leq T_1 \leq T_{1n}$. Now, as T_{2n} is the result of adding variables to T_2 that are a linear combination of variables in T_{1n} , we have $T_{2n} \leq T_{1n}$. Now, it holds that:

$$\bigoplus_{Z_j \in \mathbf{H}} Z_j = \sum_{\mathbf{H}_s \subseteq \mathbf{H}} (-2)^{|\mathbf{H}_s|-1} \prod_{Z_i \in \mathbf{H}_s} Z_i$$

$$= \sum_{\mathbf{H}_s \subseteq \mathbf{H}} (-2)^{|\mathbf{H}_s|-1} \prod_{Z_i \in \mathbf{H}_s} Z_i + (-2)^{|\mathbf{H}|-1} \prod_{Z_i \in \mathbf{H}} Z_i. \quad (\text{A.14})$$

Hence, we can express the multiplication of variables in $\mathbf{H}$ from Eq. (A.14) by separating the case of the full set $\mathbf{H}$ from the other subsets as follows:

$$\prod_{Z_j \in \mathbf{H}} Z_j = (-2)^{(|\mathbf{H}|-1)} \bigoplus_{Z_j \in \mathbf{H}} Z_j - \sum_{\mathbf{H}_s \subseteq \mathbf{H}} (-2)^{(|\mathbf{H}_s|-1)} \prod_{Z_i \in \mathbf{H}_s} Z_i. \quad (\text{A.15})$$

By recursively applying Eq. (A.15), each product term $\prod_{Z_j \in \mathbf{H}} Z_j$ can be expressed as a linear combination of XOR terms $\bigoplus_{Z_i \in \mathbf{H}_s} Z_i$, where $\mathbf{H}_s \subseteq \mathbf{H}$. This implies that any variable in T_{1n} can be written as a linear combination of variables in T_{2n} . Consequently, by an argument analogous to that in Proposition 2, we have $T_{1n} \leq T_{2n}$. Since the subsumption holds in both directions, the transformations are equivalent: $T_{1n} \equiv T_{2n}$. $\square$

Data availability

No data was used for the research described in the article.

References

- [1] J. Pearl, Probabilistic Reasoning with Intelligent Systems, Morgan & Kaufman, San Mateo, 1988.
- [2] D. Koller, N. Friedman, Probabilistic Graphical Models: Principles and Techniques, MIT Press, 2009.
- [3] R. Neapolitan, Learning Bayesian Networks, Prentice Hall, Upper Saddle River, 2004.
- [4] A. Cano, M. Gómez-Olmedo, S. Moral, Estimating conditional probabilities by mixtures of low order conditional distributions, in: International Conference on Scalable Uncertainty Management, Springer, 2017, pp. 107–118.
- [5] F.J. Díez, Parameter adjustment in Bayes networks. The generalized noisy OR-gate, in: Uncertainty in Artificial Intelligence, Elsevier, 1993, pp. 99–105.
- [6] A. Oniško, M.J. Druzdzel, H. Wasyluk, Learning Bayesian network parameters from small data sets: application of noisy-or gates, Int. J. Approx. Reason. 27 (2) (2001) 165–182.
- [7] F. Rijnmen, Bayesian networks with a logistic regression model for the conditional probabilities, Int. J. Approx. Reason. 48 (2) (2008) 659–666, in Memory of Philippe Smets (1938–2005). <https://doi.org/10.1016/j.ijar.2008.01.001>
- [8] Y. Tan, P.P. Shenoy, A bias-variance based heuristic for constructing a hybrid logistic regression-naïve Bayes model for classification, Int. J. Approx. Reason. 117 (2020) 15–28.
- [9] C. Boutilier, N. Friedman, M. Goldszmidt, D. Koller, Context-specific Independence in Bayesian networks, in: E. Horvitz, F. Jensen (Eds.), Proceedings of the 12th Conference on Uncertainty in Artificial Intelligence, Portland, Oregon, 1996, pp. 115–123.
- [10] N. Friedman, M. Goldszmidt, Learning Bayesian networks with local structure, in: Learning in Graphical Models, Springer, 1998, pp. 421–459.
- [11] A. Cano, S. Moral, A. Salmerón, Penniless propagation in join trees, Int. J. Intell. Syst. 15 (11) (2000) 1027–1059.
- [12] A.C. Restificar, T.G. Dietterich, Exploiting Monotonicity via Logistic Regression in Bayesian Network Learning, Tech. rep., School of Electrical Engineering and Computer Science, Oregon State University, 2013. https://ir.library.oregonstate.edu/concern/technical_reports/j6731495n.
- [13] A. Tharwat, T. Gaber, A. Ibrahim, A.E. Hassanien, Linear discriminant analysis: a detailed tutorial, AI Commun. 30 (2) (2017) 169–190.
- [14] A. MacAllister, A. Kohl, E. Winer, Using high-fidelity meta-models to improve performance of small dataset trained Bayesian networks, J. Mech. Des. 141 (1) (2019) 011402.
- [15] I. Deeva, A. Bubnova, Advanced approach for distributions parameters learning in Bayesian networks with Gaussian mixture models and discriminative models, Sensors 23 (1) (2023) 456, <https://doi.org/10.3390/s23010456>
- [16] S. Bengio, Y. Bengio, Taking on the curse of dimensionality in joint distributions using neural networks, IEEE Trans. Neural Netw. 11 (3) (2000) 550–558, <https://doi.org/10.1109/72.846728>
- [17] S. Lachapelle, P. Brouillard, T. Deleu, S. Lacoste-Julien, Gradient-based neural dag learning, arXiv preprint arXiv:1906.02226, 2019.
- [18] V. Kostic, G. Pacreau, G. Turri, P. Novelli, K. Lounici, M. Pontil, Neural conditional probability for uncertainty quantification, Adv. Neural Inf. Process. Syst. 37 (2024) 60999–61039.
- [19] Y. Motomura, I. Hara, Bayesian network learning system based on neural networks, in: International Symposium on Theory and Applications of Soft Computing, 2000.
- [20] B. Das, Generating conditional probabilities for Bayesian networks: easing the knowledge acquisition problem, arXiv preprint cs/0411034, 2004.
- [21] B.P. Blomaard, G.F. Nane, A.M. Hanea, Approaches for reducing expert burden in Bayesian network parameterization, Entropy 27 (6) (2025) 579.
- [22] C. Meek, D. Heckerman, Structure and parameter learning for causal Independence and extra-node models, in: UAI '97: Proceedings of the Thirteenth Conference on Uncertainty in Artificial Intelligence, Morgan Kaufmann, 1997, pp. 329–337.

- [23] K. Drury, M.J. Barons, J.Q. Smith, Surjective independence of causal influences for local bayesian network structures, arXiv preprint [arXiv:2509.24759](https://arxiv.org/abs/2509.24759), 2025.
- [24] X. Wu, K. Yue, H. Liu, L. Duan, Learning conditional probability distributions for robust probabilistic inference in Bayesian network, in: Proceedings of the 34th ACM International Conference on Information and Knowledge Management, 2025, pp. 3438–3447.
- [25] D. Heckerman, D. Geiger, D.M. Chickering, Learning Bayesian networks: the combination of knowledge and statistical data, *Mach. Learn.* 20 (1995) 197–243.
- [26] J. Quinlan, Induction of decision trees, *Mach. Learn.* 1 (1986) 81–106.
- [27] A. Cano, J.G. Castellano, A.R. Masegosa, S. Moral, Methods to determine the branching attribute in Bayesian multinets classifiers, in: Symbolic and Quantitative Approaches to Reasoning with Uncertainty: 8th European Conference, ECSQARU 2005, Barcelona, Spain, July 6–8, 2005. Proceedings 8, Springer, 2005, pp. 932–943.
- [28] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.
- [29] L. Fahrmeir, G. Tutz, *Multivariate Statistical Modelling Based on Generalized Linear Models*, vol. 425, Springer, 1994.
- [30] A. Defazio, F. Bach, S. Lacoste-Julien, Saga: a fast incremental gradient method with support for non-strongly convex composite objectives, in: Advances in Neural Information Processing Systems 27 (NIPS), Curran Associates, Inc., 2014, pp. 1646–1654, <https://papers.nips.cc/paper/5258-saga-a-fast-incremental-gradient-method-with-support-for-non-strongly-convex-composite-objectives.pdf>.
- [31] A. Dempster, N. Laird, D. Rubin, Maximum likelihood from incomplete data via the EM algorithm (with discussion), *Journal of the Royal Statistical Society, Ser. B* (1977) 1–38.
- [32] U. Fayyad, K.B. Irani, Multi-interval discretization of continuous-valued attributes for classification learning, in: Proceedings of the 13th IJCAI Conference (IJCAI' 93) Vol. 2, 1993, pp. 1022–1027.
- [33] J. Dougherty, R. Kohavi, M. Sahami, Supervised and unsupervised discretization of continuous features, in: *Machine Learning Proceedings 1995*, Elsevier, 1995, pp. 194–202, <https://doi.org/10.1016/b978-1-55860-377-6.50032-3>.
- [34] G.Y.H. Lip, R. Nieuwlaet, R. Pisters, D.A. Lane, H.J.G.M. Crijns, Refining clinical risk stratification for predicting stroke and thromboembolism in atrial fibrillation using a novel risk factor-based approach: the EURO heart survey on atrial fibrillation, *Chest* 137 (2) (2010) 263–272.
- [35] J.B. Olesen, G.Y. Lip, M.L. Hansen, P.R. Hansen, C. Tolstrup, J. Lindhardsen, C. Selmer, O. Ahlehoj, A.-M.S. Olsen, G.H. Gislason, et al., Validation of chads2, cha2ds2-vasc, and orbit-af risk scores in a cohort of patients with atrial fibrillation: a nationwide cohort study, *Thrombosis and Haemostasis* 106 (12) (2011) 1164–1171.
- [36] M. Scutari, Bayesian network repository, 2026, <https://www.bnlearn.com/bnrepository/>.
- [37] M. Scutari, Learning Bayesian networks with the bnlearn r package, *J. Stat. Softw.* 35 (i03) (2010).
- [38] A. Ankan, A. Panda, Pgmpy: probabilistic graphical models using Python, in: Proceedings of the Python in Science Conference SciPy, SciPy, 2015, pp. 6–11, <http://dx.doi.org/10.25080/Majora-7b98e3ed-001>.
- [39] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: machine learning in Python, *J. Mach. Learn. Res.* 12 (2011) 2825–2830.
- [40] T. Roos, H. Wettig, P. Grünwald, P. Myllymäki, H. Tirri, On discriminative Bayesian network classifiers and logistic regression, *Mach. Learn.* 59 (2005) 267–296.
- [41] A. Ankan, A. Panda, Pgmpy: probabilistic graphical models using Python, in: Proceedings of the 14th Python in Science Conference (SCIPY 2015), Citeseer, 2015, pp. 6–11.